Applied Soft Computing Journal 185 (2025) 113970

Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc

# Deadline-constrained and bi-objective workflow scheduling with fuzziness in cloud computing systems

Jie Zhu[a,*], Fengmei Liu[a], Jingzhe Sun[a], Haiping Huang[a,b]

[a] School of Computer Science, Nanjing University of Posts & Telecommunications, Nanjing, 210036, Jiangsu, China
[b] Jiangsu High Technology Research Key Laboratory for Wireless Sensor Networks, Nanjing University of Posts & Telecommunications, Nanjing, 210036, Jiangsu, China

## HIGHLIGHTS

- A bi-objective and fuzzy workflow scheduling problem is formulated with uncertain processing times, uncertain transmission times, uncertain due dates, nonlinear dependency, and multiple price modes for scalable resources in clouds. The uncertain parameters are represented by fuzzy triangular numbers are considered.
- Linked list structures are introduced to manage the fuzzy idle time slots of VMs, where time slots are linked in ascending order of the start time. The feasible fuzzy time slots can be quickly located and assigned to workflow instances.
- An iterative heuristic integrating simulated annealing and various neighborhood searches is proposed to obtain the fuzzy near-Paretooptimal schedules with delicately designed fuzzy temporary objectives. These temporary objectives are highly related to the objectives to optimize in real schedules.

## ARTICLE INFO

*Keywords:*
Cloud workflow
Fuzziness
Pareto
Multi-objective
Cloud computing

## ABSTRACT

Deploying intelligent expert systems on the cloud is a cost-effective solution for handling compute-intensive service requests, where each request is treated as a workflow processing procedure. Cloud workflows break down complex service requests into smaller tasks and take advantage of automatic maintenance services provided by the cloud workflow management system (CWMS). CWMS can orchestrate task execution, handle dependencies, and direct dynamic scaling of resources based on workload demands. For a CWMS, the main challenge lies in the uncertainty of workflow scheduling, i.e., the processing time, the data transmission time, and the due date are not crisp values. This paper investigates the problem of bi-objective workflow scheduling under fuzziness, aiming to minimize both the total rental cost and the degree of user dissatisfaction. Triangular fuzzy numbers are used to represent the uncertainty of temporal parameters. Two general pricing models in cloud systems are considered: on-demand and reserved price structures. A bi-objective fuzzy workflow scheduling framework is proposed, which consists of workflow sequencing, fuzzy solution generation and solution improvement components. The workflow sequencing component determines the priorities of the workflows. The fuzzy solution generation component assigns tasks to appropriate resources. The Simulated Annealing with Variable Neighborhood Search (SAVNS) method is developed for the solution improvement component. The experimental results demonstrate that the proposal can achieve better effectiveness and robust performance than the baseline algorithms compared. The proposed method can offer practical solutions for CWMS to optimize cost-performance trade-offs in uncertain environments.

## 1. Introduction

An intelligent expert service can be divided into multiple stages to run in parallel. For example, in a Human action recognition service that assists Physical Therapy [1], a patient video taken at home should be pre-processed by several stages before the clinician's review: video filtering, video segmenting, face recognition, head rotation detection, hand pose recognition, leg movement recognition, event prejudging, etc. These stages are dependent. The dependencies among these stages are usually nonlinear, i.e., some stages can be processed in parallel (e.g., movement detection for head, hands and legs) whereas some stages should be performed sequentially (e.g., video filtering and segmenting).

* Corresponding author.
*Email address:* zhujie@njupt.edu.cn (J. Zhu).

https://doi.org/10.1016/j.asoc.2025.113970
Received 3 February 2025; Received in revised form 10 August 2025; Accepted 20 September 2025
Available online 23 September 2025
1568-4946/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Most of these stages require intensive computing resources since video processing usually involves graphics processing unit (GPU) computing. Therefore, a pre-processing service of a patient video can be considered as a compute-intensive workflow instance with a directed acyclic graph (DAG) structure. It is difficult to obtain the accurate processing time of these stages in advance. We can only estimate the processing time of these stages based on empirical statistics. Due to economic and technological reasons, it is a costly task to complete massive compute-intensive workflow instances on local infrastructures. Many workflow service providers [2] prefer to deploy their applications in clouds and take advantage of the automatic maintenance service provided by cloud workflow management systems (CWMS).

CWMS is one of the key systems provided by many public cloud platforms (e.g., Microsoft Azure and Amazon Web Services). To achieve complete maintenance of the life cycle, a workflow service must be implemented according to the development specifications and guidelines provided by the public cloud platforms. Then, the CWMS is responsible for provisioning and maintaining the service. Since different stages of the workflow service have different computing resource requirements, a general practice of CWMS is to prepare multiple virtual clusters of computing resources, where the resources in each virtual cluster are configured specifically for only one certain stage of workflow instances [3]. For cost considerations, a virtual cluster is initially provisioned with reserved virtual machines (VMs), since reserved VMs are charged with a significant discount. When CWMS receives requests for the service: (i) it formulates each request into a DAG-based workflow instance; (ii) it estimates the task processing in the workflow instances; (iii) it assigns due dates to workflow instances according to the demands submitted along with the requests; (iv) it schedules instances on suitable VMs; (v) it scales out the overloaded virtual clusters by temporarily renting on-demand VMs. As can be seen, scheduling workflow instances feasibly, effectively, and efficiently is the major challenge for CWMS.

In the paper, the problem of bi-objective cloud workflow scheduling with fuzziness is studied. The problem involves multiple computing-intensive requests, where each request consists of multiple stages to process. Each request is formulated as a workflow instance. Each workflow instance should be completed before its due date. The due date of an instance is determined by CWMS according to the user's demand and some user patience model [4]. The processing time, data transmission time and due date are uncertain and fuzzy. Several virtual clusters are configured to handle these workflow instances. Each virtual cluster is configured to process a certain stage of the workflow instances. VMs in the same virtual cluster are charged in either reserved mode or on-demand mode. Reserved VMs are rented for a long term, so they are less expensive. On-demand VMs are charged hourly or even minutely. They are more expensive than reserved ones. We aim to optimize two objectives in the problem under study: the average satisfaction degree and the total rental cost.

The problem with fuzzy parameters is more practical than the one with crisp temporal parameters [5], which is NP-hard. Therefore, the problem with fuzziness is also NP-hard. There are three major challenges in the problem under study. (i) Task processing time, task transmission time and workflow due date are fuzzy and uncertain. Fuzzy temporal parameters lead to fuzzy assignments of tasks, which make idle time slots in virtual clusters fuzzy and discrete. Managing fuzzy time slots in a virtual cluster and generating feasible schedules require longer computing time compared to those with crisp values. In addition, the scheduling process is complex due to the non-linear dependencies among stages of workflows. (ii) Reserved VMs in each virtual cluster are limited. It is very possible that these VMs are not capable of handling very heavy workloads, which leads to many overdue workflow instances and low user satisfaction. In order to gain higher user satisfaction, CWMS rents on-demand VMs to alleviate the overloaded virtual clusters. However, more on-demand VMs may result in a higher rental cost. Minimizing rental costs and maximizing user satisfaction are two conflicting objectives. It is not easy to strike a balance. (iii) The total cost and the average degree of satisfaction cannot be obtained directly from a fuzzy schedule that is only a predictive plan. Workflow instances are executed according to the VM assignments and stage orders in the fuzzy schedule. Only when all the workflow instances are completed can the real schedule be obtained and evaluated by the objectives. How to evaluate the fuzzy schedules obtained in the process of optimization and select the best one to execute are the big challenges. Delicately designing temporary fuzzy objectives is critical for improving the fuzzy schedules, which further leads to the real optimized schedule.

We aim to provide an effective solution to the problem under study. In this paper, we make the following contributions.

(i) A bi-objective optimization model is presented for cloud workflows. The model simultaneously addresses temporal uncertainty (in task processing times, data transmission delays, and deadlines) and hybrid pricing models (reserved vs. on-demand VMs). Triangular fuzzy numbers (TFNs) are used to capture real-world unpredictability, enabling robust scheduling under ambiguity.
(ii) A Simulated Annealing with Variable Neighborhood Search (SAVNS) is designed to search for the fuzzy and near-optimal solutions with delicately designed fuzzy temporal objectives. SAVNS consists of three key components: the component to determine the priorities of workflows, the component to generate feasible timetables for a given workflow sequence and the component to effectively search in the solution space.
(iii) Two idle time-slot management strategies are designed for virtual clusters, optimizing VM utilization by prioritizing either the earliest start times or the shortest slot durations. With these strategies, a task can be quickly assigned to the time slot with an earlier start time or to the time slot with a shorter duration.

The remainder of the paper is organized as follows. In Section 2, the related studies are reviewed. The problem under study is formulated in Section 3. In Section 4, the proposed algorithm is presented. The performance of the proposed algorithm is comprehensively evaluated in Section 5. Finally, conclusions and future research are provided in Section 6.

## 2. Related work

Many CWMSs have successfully implemented the automatic operation and maintenance of cloud services, such as Apache Airavata [6], Pegasus [7] and Swift [8]. Users are relieved of the details of the service management, such as where and how their requests are processed. A CWMS is responsible for making decisions about resource provisioning and task offloading. Therefore, the scheduling mechanism used by CWMS has a significant impact on the performance of the system.

Workflow scheduling problems with deadline constraints in the cloud environment are the most thoroughly investigated problems. The deadline constraints in these studies are "hard" or "tight", which means that no violation is allowed. Abdi et al. [9] consider the security requirements and data dependency in the cost-efficient workflow scheduling problem. They propose a mixed-integer linear programming model with the objectives of minimizing execution cost, while satisfying the security requirements and deadline constraints. Yang et al. [10] formulate the workflow scheduling in clouds and aim to optimize execution cost with a deadline constraint. A hybrid scheduling algorithm is proposed, where a heuristic called Cloud Partial Critical Paths and Particle Swarm Optimization are integrated to solve the considered problem. Ye et al. [11] design a dynamic scheduling algorithm to schedule stochastic multi-workflows with a deadline constraint. Later on, they introduce a cost-driven algorithm for a similar workflow scheduling problem in cloud computing [12]. The cost-driven algorithm combines ant colony optimization (ACO) with cost-driven heuristics. Taheri et al. [13] study the problem of scheduling periodic and time-constrained scientific workflows. A cloud broker is presented, which consists of an off-line planner

and dynamic scheduling components. The off-line planner plans the number of reserved resources to rent. The dynamic scheduler leases on-demand resources for heavy workloads. Rajput et al. [14] investigate Spark workflow scheduling in multi-cloud networks. A novel heuristic-based approach is developed to obtain solutions that meet different constraints such as the heterogeneity of VM instances, priority constraints, transmission times, and the impact of performance uncertainty.

Most of the studies mentioned above take the cost as the primary objective to optimize. The special pricing models in clouds make optimization more complex. Li et al. [15] study cloud workflow applications considering varying electricity prices. They present an adaptive local search method to dynamically balance intensification using neighborhood structures of increasing size. Sun et al. [16] present a hybrid heuristic called enhanced task type first algorithm for cloud workflow scheduling with hibernation and per-second billing. Lin et al. [17] investigate the scheduling problem for Function-as-a-Service (FaaS) applications. They develop a profile performance and cost modeling algorithm to trade-off between the performance and cost of FaaS applications.

Makespan, energy consumption, reliability, and utilization are also commonly optimized objectives in many workflow scheduling problems. Wu et al. [18] try to optimize makespan and cost for cloud workflow scheduling simultaneously. A multi-objective evolutionary list scheduling algorithm is proposed to address it. Qin et al. [19] explore the cloud workflow scheduling problem with three objectives: execution time, cost and risk. Li et al. [20] develop a scoring and dynamic hierarchy-based non-dominated sorting genetic algorithm II (NSGA-II) to optimize makespan and cost for executing workflows. Wu et al. [21] design a practical scheduling mechanism with the aim of minimizing makespan and rental cost among multiple cloud service providers. Guo et al. [22] optimize three objectives (cost, makespan and energy consumption) in hybrid cloud workflow scheduling. They design a grouping strategy with dynamic neighborhood. Zhu et al. [23] develop a pointer network-based asynchronous dominant actor-critic (A3C) scheduling method to jointly optimize energy consumption and latency. Gao et al. [34] tackle the NP–hard Travelling Salesman Problem (TSP) by proposing an algorithm. It combines Farmland Fertility Algorithm, a hyper-heuristic based on Modified Choice Function, and Lin-Kernighan local search to find the shortest Hamiltonian route.

Involving uncertainty, time-varying parameters, or dynamic cloud characteristics in cloud workflow scheduling is very popular in recent topics about cloud computing. Zhu et al. [5] consider a problem scenario where workflows arrive stochastically, task processing time is uncertain and due dates of workflows are fuzzy. They employ a triangular fuzzy number (TFN)-based model to formulate the problem model. An iterative heuristic is developed to schedule tasks periodically. Zhang et al. [24] transform uncertain factors in the cloud task scheduling problem into interval parameters. They simultaneously optimize cost, makespan, workload balance, and completion rate of tasks. An evolutionary algorithm with many objectives is employed to address the problem. Rizvi et al. [25] propose an intelligent fuzzy method to offload tasks with uncertain parameters to appropriate resources. The uncertain parameters refer to the computation time, the deadline and the communication overhead. They present a salp swarm algorithm to learn fuzzy allocation rules and obtain high-quality solutions. Anka et al. [32] review the Artificial Rabbit Optimization (ARO) algorithm. It mimics rabbit behavior, has multiple advantages, and is applied to solve complex problems. Gharehchopogh et al. [33] aim to analyze the Manta Ray Foraging Optimization (MRFO) algorithm. They use the MRFO algorithm, inspired by manta rays' foraging behaviors, to deal with complex optimization problems. Zhang et al. [35] deal with cloud task scheduling by turning uncertain factors into interval parameters and optimizing multiple objectives like cost and makespan using an evolutionary algorithm. Zhang et al. [36] attempt to enhance the performance of the Levy-flight firefly algorithm(FA) by proposing an adaptive logarithmic spiral-Levy FA to strengthen local exploitation and accelerate convergence for global continuous optimization problems. Chen et al. [26] study multiple workflow applications in fuzzy edge-cloud computing environments. The computing capacity of the servers and the bandwidth are uncertain. They employ TFNs to describe these uncertain parameters. The quadratic penalty function is integrated into the particle swarm optimization for the problem under study.

In summary, the related literature can be classified into several categories based on the attributes of the problems considered, such as the deadline constraint, multiple objectives, multiple pricing modes, and uncertain factors. Table 1 shows the comparison of studies in literature. Most of the existing literature only considers combinations of two or three of the attributes mentioned above. To our knowledge, few studies consider all the above dynamic cloud characteristics.

## 3. Problem description

Three roles are involved in the considered problem: CWMS, cloud service provider (CSP) and end user. CWMS receives the request for CSP service deployment and provides multiple virtual clusters according to the DAG structure described in the service topology. When the service is available, the end users submit requests for the service and CWMS processes the service requests. CWMS formulates service requests as workflow instances and estimates temporal information (task processing time, intermediate data transmission time and workflow due date) for each instance. Based on estimated information and the user patience model [4], CWMS assigns due dates to workflow instances. Then CWMS generates the schedule according to the VM availability and the temporal information of the workflow instances. The schedule indicates the allocated VM and the priority for each task. It also determines the rental plan for new VMs, in case the currently available VMs in some virtual clusters are not sufficient. Finally, CWMS allocates all tasks to VMs and monitors them. Based on the actual execution of the workflow instances, CWMS evaluates the schedule with respect to user satisfaction and rental cost. In the paper, the arrival times of the workflow instances are given and are certain. Task preemption is not allowed. We do not consider multiple thread scenarios.

There are $n$ workflow instances $W_1, \dots, W_n$ to schedule in the problem considered. We use $\mathbb{W}$ to represent the set of workflow instances. CWMS offloads these instances to $m$ virtual clusters. We use $VC_i$ to denote the $i^{th}$ virtual cluster ($i = 1, \dots, m$). The dependencies in a workflow instance are described as $\mathbb{G} = \langle \mathbb{V}, \mathbb{E} \rangle$. $\mathbb{V}$ consists of $m$ stages and $\mathbb{E}$ includes dependencies between stages. A stage of a workflow instance is a task to execute. Therefore, there are $m$ tasks to execute in a workflow instance. We use $t_{j,i}$ to denote the $i^{th}$ task of $W_j$. $Pt_{j,i}$ represents the immediate predecessor set of task $t_{j,i}$. $Pt_{j,i} = \{t_{j,i'} | < t_{j,i'}, t_{j,i} > \in \mathbb{E}\}$. $St_{j,i}$ denotes the immediate successor set of task $t_{j,i}$. $St_{j,i} = \{t_{j,i'} | < t_{j,i}, t_{j,i'} > \in \mathbb{E}\}$. The dummy source task $t_{j,0}$ and the dummy sink task $t_{j,n+1}$ are defined for $W_j$.The notations used are listed in Table 2.

We employ triangular fuzzy numbers to describe the fuzzy temporal parameters in the problem. A TFN $\widetilde{x}$ can be represented by $\widetilde{x} = (x^1, x^2, x^3)$, where $x^1 \le x^2 \le x^3$. $x^1$, $x^2$ and $x^3$ are the minimal value, the most probable value and the maximal value, respectively. The addition, multiplication, max, and ranking operations of TFN are defined in [5]. Three fuzzy temporal parameters are the task processing time $\widetilde{p}_{j,i}$, the data transmission time $\widetilde{d}_{j,i,i'}$ and the due date $\widetilde{D}_j$ of $W_j$. $\widetilde{d}_{j,i,i'}$ denotes the transmission time of data between $t_{j,i}$ and $t_{j,i'}$ ($t_{j,i} \in Pt_{j,i'}$). The real processing time and data transmission time of a task are not known before the task is finished. We use $p_{j,i}$ and $d_{j,i,i'}$ to denote the real values of $\widetilde{p}_{j,i}$ and $\widetilde{d}_{j,i,i'}$, respectively.

A virtual cluster $VC_i$ ($i = 1, \dots, m$) is configured specifically for tasks of $i^{th}$ stage. The VMs provisioned in the same virtual cluster are homogeneous in computing capabilities. Two types of VMs are provisioned in a virtual cluster according to the price modes. We use sets $R_i$ and $O_i$ to contain the reserved VMs and the on-demand VMs, respectively. The number of reserved VMs in a cluster is fixed, but the number of on-demand VMs varies with time. Initially, there are no on-demand

**Table 1**
Literature comparison.

| Ref. | Constraint | Objective | Uncertain factor | Algorithm |
|---|---|---|---|---|
| [9] | Deadline, security requirements | Cost | N/A | Mixed−Integer linear programming (MILP) |
| [10] | Deadline, task dependencies | | | Hybrid algorithm |
| [12] | | | | F−ant colony optimization (F−ACO) |
| [15] | | | | Energy−aware cloud workflow scheduling with geo−distributed data (ECWSD) |
| [11] | Deadline, task dependencies | Cost | Task execution time | Stochastic Multi−Workflows dynamic scheduling algorithm (SMWDSA) |
| [13] | | | | Hybrid Algorithm |
| [14] | Deadline, task dependencies | Cost | VM performance, transmission time | Dynamic scheduling approach for Spark Workflow Tasks Scheduler (DSWTS) |
| [16] | Deadline, task dependencies, resource availability | Cost,total idle rate | N/A | Enhanced Task Type First Algorithm (ET2FA) |
| [17] | Task dependencies, execution and network delays | Cost, response time | Task execution time, workflow branching | First Bottleneck Alleviation (DFBA) |
| [18] | Task dependencies, data transmission | Cost, makespan | N/A | Multiobjective Evolutionary List Scheduling (MOELS) algorithm |
| [21] | Deadline, task dependencies | | | Multi−objective multi−workflow scheduling mechanism (MOMWS) |
| [19] | Task dependencies, security level, risk constraints | Cost, makespan, risk | N/A | Cluster−based Cooperative Co−evolutionary Algorithm (CBCC) |
| [20] | CPU capacity, bandwidth | Cost, makespan | N/A | Scoring and Dynamic Hierarchy−based NSGA−II (SDHN) |
| [23] | Task dependencies, data locality, security | | Task sequences | Hybrid Algorithm |
| [22] | Task dependencies, data locality constrains, deadline | Cost, makespan, energy consumption | N/A | Workflow Dynamic Neighborhood Search (WDNS) algorithm |
| [24] | Task dependencies, network bandwidth | Cost, makespan | Network bandwidth and MIPS | Interval Many−Objective Evolutionary Algorithm (InMaOEA) |
| [25] | Deadline, task dependencies | Cost, makespan | Computation time, communication overhead, deadlines | Intelligent Salp Swarm Scheduler with Fitness−based Quasi−Reflection method (ISSS−FQR) |
| [26] | | Cost | Computing capacity, bandwidth | Scheduling Strategy based on Particle Swarm Optimization algorithm employing the Quadratic Penalty Function (SSPSO−QPF) |
| Our proposal | Deadline, task dependencies | Cost, satisfaction degree | Processing time, due date, transmission time | Simulated Annealing with Variable Neighborhood Descent (SAVND) |

**Table 2**
Notations used for the problem description.

| Notation | Description |
|---|---|
| $\mathbb{G} = \langle \mathbb{V}, \mathbb{E} \rangle$ | DAG structure |
| $\mathbb{V}$ | Set of stages in the workflow |
| $\mathbb{E}$ | Precedence dependency set |
| $\mathbb{W}$ | Set of workflow instances |
| $n$ | Number of workflow instances |
| $W_j$ | $j^{th}$ workflow instance, $j = 1, \ldots, n$ |
| $t_{j,i}$ | $i^{th}$ task of $W_j$, $i = 1, \ldots, m$ |
| $m$ | Number of stages in the workflow |
| $Pt_{j,i}$ | Set of immediate predecessors of $t_{j,i}$ |
| $St_{j,i}$ | Set of immediate successors of $t_{j,i}$ |
| $\widetilde{p}_{j,i}$, $p_{j,i}$ | Fuzzy/real processing time of $t_{j,i}$ |
| $\widetilde{d}_{j,i,i'}$,$d_{j,i,i'}$ | Fuzzy/real transmission time |
| $\widetilde{b}_{j,i}$, $b_{j,i}$ | Fuzzy/real start time of $t_{j,i}$ |
| $\widetilde{c}_{j,i}$, $c_{j,i}$ | Fuzzy/real completion time of $t_{j,i}$ |
| $\widetilde{D}_j$ | Fuzzy due date of $W_j$ |
| $VC_i$ | Virtual cluster for $i^{th}$ stage |
| $\psi_i^r$ | Unit price for reserved VMs in $VC_i$ |
| $\psi_i^o$ | Unit price for on-demand VMs in $VC_i$ |
| $u$ | Charging time unit |
| $R_i$ | Reserved VM set in $VC_i$ |
| $O_i$ | On-demand VM set in $VC_i$ |
| $\widetilde{s}_{j,i}$, $s_{j,i}$ | Fuzzy/real assignment of $t_{j,i}$ |
| $v_{j,i}$ | VM assigned to $t_{j,i}$ |
| $\widetilde{F}$, $F$ | Fuzzy/real schedule |

VMs. On-demand VMs will be rented later according to the schedule generated by CWMS. VMs are charged by the time unit $u$. The unit price of the on-demand VM is $\psi_i^o$ and the one for a reserved VM is $\psi_i^r$. Generally, on-demand VMs are more expensive than reserved VMs. When an on-demand VM is idle, it will be released.

An assignment of task $t_{j,i}$ is defined as $\widetilde{s}_{j,i} = \langle \widetilde{b}_{j,i}, \widetilde{c}_{j,i}, v_{j,i} \rangle$. $v_{j,i}$ is the VM to which task $t_{j,i}$ is assigned. $t_{j,i}$ starts at time $\widetilde{b}_{j,i}$ and ends at time $\widetilde{c}_{j,i}$. Fig. 1 (a) depicts the Gantt chart of a fuzzy assignment. Once a task is completed, its fuzzy assignment can become a real one, i.e., $s_{j,i} = \langle b_{j,i}, c_{j,i}, v_{j,i} \rangle$. We need to determine the assignments for all tasks in all workflow instances. A fuzzy schedule solution $\widetilde{F}$ is the set that contains all fuzzy assignments, and a real schedule solution $F$ contains all real ones. $\widetilde{F}$ is feasible only when it satisfies the following constraints.

- The non-preemption constraint indicates that a task cannot be interrupted during execution.

$$\widetilde{c}_{j,i} = \widetilde{b}_{j,i} + \widetilde{p}_{j,i} \tag{1}$$

- The precedence constraint implies that a task can start after receiving the intermediate data from all its predecessors:

$$\widetilde{b}_{j,i} \geq \max\{\widetilde{c}_{j,i'} + \widetilde{d}_{j,i',i} | t_{j,i'} \in Pt_{j,i}\} \tag{2}$$

- The non-conflict constraint means that tasks on the same VM cannot conflict. Given two fuzzy assignments $\widetilde{s}_{j,i}$ and $\widetilde{s}_{j',i}$ in the same VM:

$$\max\{\widetilde{b}_{j,i}, \widetilde{b}_{j',i}\} \geq \min\{\widetilde{c}_{j,i}, \widetilde{c}_{j',i}\} \tag{3}$$

- The fuzzy deadline constraint guarantees that the fuzzy completion time of a workflow instance cannot exceed its fuzzy due date (the completion time $\widetilde{c}_{j,m+1}$ of the dummy sink task $t_{j,m+1}$ is the completion time of $W_j$).

$$\widetilde{c}_{j,m+1} \leq \widetilde{D}_j \tag{4}$$

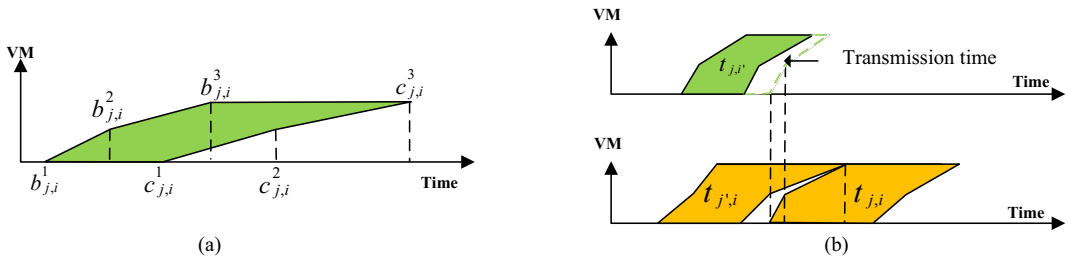



**Fig. 1.** Fuzzy assignment.

Fig. 1(b) shows an example of calculating a feasible fuzzy assignment $s_{j,i}$ for $t_{j,i}$ given the fuzzy assignments of its predecessor task $t_{j,i'}$ and its previous task $t_{j',i}$ on the same VM.

A feasible real schedule is also restricted to the above-mentioned constraints except for the deadline constraint. It cannot guarantee the hard deadline constraint since the real completion time of tasks cannot be precisely determined in advance. It is possible that a workflow instance is not completed before its assigned due date. Therefore, the deadline constraint considered in the paper is the soft one, i.e., it is guaranteed in the fuzzy schedule but could be violated in the real one.

The fuzzy schedule $\widetilde{F}$ is a predictive solution. All tasks in workflow instances are executed following the task sequence on each VM in a fuzzy schedule $\widetilde{F}$. Fuzzy assignments are replaced by real assignments (completed) as time goes by. The real schedule $F$ is obtained when all assignments are completed. Fig. 2 shows the Gantt chart of a fuzzy schedule where tasks assigned to $VC_1$ are predecessor tasks of those assigned to $VC_2$. As can be seen in Fig. 2, the fuzzy schedule has been partially turned into a real state. $t$ is the current time and the assignments before $t$ have already become the real assignments. Once the real schedule $F$ is obtained, we evaluate its quality based on aspects of the degree of dissatisfaction $\overline{DS}$ and the total rental cost $C$.

The degree of dissatisfaction $DS_j$ is calculated based on the completion time of the workflow $c_{j,m+1}$ and the due date $\widetilde{D}_j$ (as shown in Fig. 3 and Eq. (5)). The average degree of dissatisfaction $\overline{DS}$ can be computed by Eq. (6).

$$DS_j = \begin{cases} 0, & c_{j,m+1} \le D_j^2 \\ \frac{c_{j,m+1}-D_j^2}{D_j^3-D_j^2}, & D_j^2 < c_{j,m+1} < D_j^3 \\ 1, & c_{j,m+1} \ge D_j^3. \end{cases} \tag{5}$$

$$\overline{DS} = \frac{1}{n}\sum_{j=1}^{n} DS_j \tag{6}$$

$C$ is the total rental cost and can be calculated by Eqs. (7)–(10). $C$ consists of two parts: the rental cost $C_r$ for all reserved VMs and the rental cost $C_o$ for all VMs on demand. $C_r$ can be computed on the basis of the completion time $\mathcal{T}_{max}$ of a schedule (Eq. (8)). The rental cost of an on-demand VM can be computed on the basis of $\mathcal{T}_v$ by Eq. (11).

$$C = C_r + C_o \tag{7}$$

$$C_r = \lceil \frac{\mathcal{T}_{max}}{u} \rceil \times \psi_i^r \times \sum_{i=1}^{m} |R_i| \tag{8}$$

$$\mathcal{T}_{max} = \max_{j=1}^{n}\{c_{j,m+1}\} \tag{9}$$

$$C_o = \sum_{i=1}^{m}\sum_{v\in O_i} \lceil \frac{\mathcal{T}_v}{u} \rceil \times \psi_i^o \tag{10}$$

$$\mathcal{T}_v = \max\{c_{j,i}|v_{j,i} = v\} - \min\{c_{j,i}|v_{j,i} = v\}$$
$$i = 1,\dots,m; j = 1,\dots,n \tag{11}$$

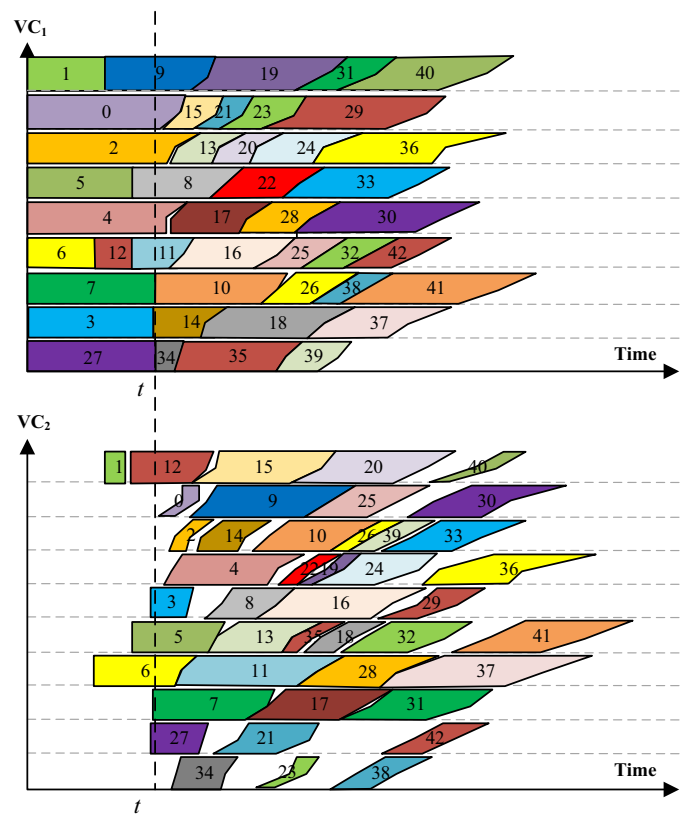



**Fig. 2.** Gantt chart for a fuzzy schedule.

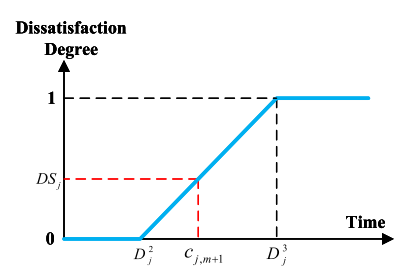



**Fig. 3.** Dissatisfaction degree.

The paper aims to produce fuzzy feasible schedules that may lead to Pareto-optimal solutions with the minimum degree of dissatisfaction and total rental cost. The two objectives might conflict to some extent.

A lower degree of dissatisfaction means that more on-demand VMs are required to complete the workflow instances as soon as possible, which may lead to higher rental costs for on-demand VMs. Renting

fewer on-demand VMs can reduce the cost; however, it may cause a very tight schedule, i.e., the workflow completion times being very close to the due dates. A tighter fuzzy schedule means that more workflow instances may be completed after their due dates. From the perspective of multi-objective optimization, therefore, the proposed algorithm should perform well in terms of proximity and diversity.

## 4. Proposed algorithm

In the paper, a bi-objective and iterative heuristics is proposed for fuzzy cloud workflow scheduling. The algorithm framework is given in Algorithm 1.

Firstly, in the framework, the Workflow Sequencing (WS) component is introduced to determine the priorities of the workflow instances (Step 1). Then the Fuzzy Solution Generation Method (FSGM) component generates the initial fuzzy schedule $\widetilde{F}^0$ based on the workflow sequence $\pi^0$ (Step 2). In FSGM, ready tasks are sorted and allocated to available fuzzy time slots on VMs, and VM availability is updated accordingly. $\widetilde{F}^0$ is improved by the Simulated Annealing algorithm with Variable Neighborhood Search (SAVNS) and a fuzzy Pareto set $\widetilde{PF}$ is obtained (Step 3). Finally, we execute all fuzzy solutions in the Pareto set and obtain the real Pareto set $PF$ (Steps 4–8). In reality, only one fuzzy schedule in $\widetilde{PF}$ should be executed and the other ones should be discarded. When $\widetilde{PF}$ is obtained, we can select one schedule according to the system preference. The execution of the selected schedule is monitored and the crisp values of temporal information are determined. Since the considered problem is not a dynamic scheduling problem, the monitor crisp values are only used in the final evaluation of the generated schedules. However, evaluating the real Pareto set $PF$ can show whether the proposed algorithm is capable of producing solutions uniformly distributed and provides various solutions of different preferences for CWMS to choose.

In the above framework, the Workflow Sequencing (WS) component determines the priorities of workflow instances and outputs the workflow sequence according to the priorities. The FSGM component assigns tasks to appropriate resources. The SAVNS based heuristics try to search for the optimal solutions. These key components are further introduced in the following sections.

### 4.1. Workflow sequencing

Two workflow sequencing strategies are introduced:

- Earliest Due Date First (EDDF) strategy sequences workflow instances in ascending order by their fuzzy due dates.
- Smallest Slack Time First (SSTF) [15] strategy sequences workflow instances in ascending order by their fuzzy slack time.

The fuzzy slack time $\widetilde{\mathcal{T}}_j^{slack}$ of $W_j$ is defined by Eq. (14), where $\widetilde{c}^*_{j,m+1}$ is the fuzzy earliest completion time of $W_j$. Slack time is the extra time reserved in the schedule to address potential delays. It is also known as buffer time, elastic time, or backup time.

$$\widetilde{c}^*_{j,0} = 0 \tag{12}$$

$$\widetilde{c}^*_{j,i} = \max\{\widetilde{c}^*_{j,i'} + \widetilde{d}_{j,i',i} | t_{j,i'} \in Pt_{j,i}\} + \widetilde{p}_{j,i} \tag{13}$$

$$\widetilde{\mathcal{T}}_j^{slack} = \widetilde{D}_j - \widetilde{c}^*_{j,m+1} \tag{14}$$

**Algorithm 1:** Bi-objective fuzzy workflow scheduling framework (bi-FWSF).

1 Perform Workflow Sequencing to generate initial workflow sequence $\pi^0$;
2 Perform Fuzzy Solution Generation Method to generate the initial fuzzy solution $\widetilde{F}^0$ for $\pi^0$;
3 Perform Simulated Annealing with Variable Neighborhood Descent Heuristics to improve $\widetilde{F}^0$ and generate the fuzzy Pareto set $\widetilde{PF}$;
4 $PF \leftarrow \emptyset$;/* Real Pareto set */
5 **for** $\forall \widetilde{F} \in \widetilde{PF}$ **do**
6 Monitor the execution of $\widetilde{F}$ and determine crisp values of task processing time and data transmission time;
7 Generate real schedule $F$;
8 Put $F$ into $PF$ if $F$ is a non-dominated solution;
9 **return** $PF$;

### 4.2. Fuzzy solution generation method

A workflow sequence indicates the priorities of the workflow instances. Different workflow sequences may lead to different schedules. FSGM computes a feasible fuzzy schedule for a given sequence (Algorithm 2). Firstly, in FSGM, all ready tasks are selected and placed in the set $PQ$ (Line 1). If the predecessor of a task is the dummy source task, then the task is ready to start. Secondly, ready tasks are prioritized according to the priorities of their workflow instances in the given $\pi$ (Line 2). Thirdly, the iteration is repeated until the assignments of all ready tasks are obtained (Lines 3–9). In each iteration, a suitable VM is selected for the task with the highest priority (Line 4). The fuzzy assignment of the task is computed (Line 6). The availability of the VM is updated accordingly (Line 6). The ready task is removed from $PQ$ (Line 7) and new ready tasks are placed in (Line 8). At the moment, a task is ready to schedule only if all its predecessor tasks are scheduled. Finally, the priorities of ready tasks are calculated (Line 9).

There are three main components in FSGM. VM Availability Maintenance (VMAM) component is designed to collect idle time slots for every virtual cluster. The task allocation (TA) component is developed to select a suitable virtual machine for a given ready task. The task prioritization (TP) component is used to compute the ready task priorities.

#### 4.2.1. VM availability maintenance

The idle time on a VM is fragmented into many discrete time slots during the task allocation process. The idle slots of VMs in the same virtual cluster should be organized to achieve quick task assignment. In VMAM, the data structure called linked list is employed to arrange idle slots. An idle slot is stored as a node in the linked list of the corresponding virtual cluster. Two linked lists are managed for each virtual cluster. The idle slots of the on-demand VMs in $VC_i$ are linked in the linked list $\mathbb{L}_i^o$. The idle slots of the reserved VMs in $VC_i$ are linked in the other linked list $\mathbb{L}_i^r$. We use two strategies to build a linked list for a given virtual cluster.

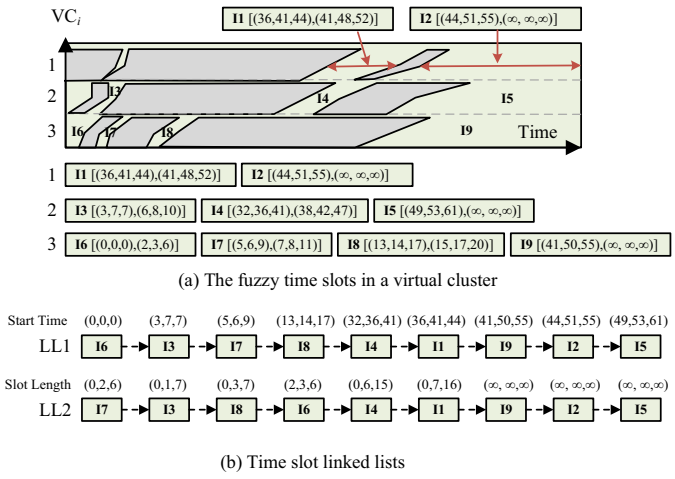
(1) LL1 links the nodes in ascending order of the start times of slots, i.e., the earliest idle slot comes first on the link.

(2) LL2 links the nodes in ascending order of the lengths of slots, i.e., the shortest idle slot comes first on the link.

Fig. 4 shows an example of VMAM. Given the idle time slots in one virtual cluster Fig. 4(a), they are stored in the linked list structures as shown in Fig. 4(b).

**Algorithm 2:** Fuzzy solution generation method FSGM ($\pi$).

1 Find all ready tasks and put them in set $PQ$;
2 Call Task Prioritization to compute priorities of ready tasks based on $\pi$;
3 **while** $PQ \neq \emptyset$ **do**
4 Perform Task Allocation to assign the task $t^*_{j,i} \in PQ$ with the highest priority to an appropriate VM $v$;
5 Calculate $\widetilde{s}^*_{j,i}$ for $t^*_{j,i}$;
6 Call VM Availability Maintenance to update availability of VM $v$;
7 $PQ \leftarrow PQ - \{t^*_{j,i}\}$;
8 Check if successor tasks of $t^*_{j,i}$ are ready and put the ready ones into $PQ$;
9 Call Task Prioritization to compute priorities of ready tasks;
10 **return** $\{\widetilde{s}^*_{j,i} | j = 1, \ldots, m; i = 1, \ldots, n\}$.




(a) The fuzzy time slots in a virtual cluster

(b) Time slot linked lists

**Fig. 4.** Maintenance of idle time slots in one virtual cluster.

### 4.2.2. Task allocation

Given a ready task $t_{j,i}$ to schedule, TA first searches the linked list $\mathbb{L}_i^r$ for the time slot that is feasible to accommodate the task. If no feasible time slot on $\mathbb{L}_i^r$ is found, TA will try the linked list $\mathbb{L}_i^o$. If not yet a feasible time slot can be found, a new on-demand VM will be rented and added to the cluster. In the new VM, $t_{j,i}$ should start as soon as possible.

If LL1 is applied to construct the linked list of idle time slots, the slot with the earliest start time will be assigned to $t_{j,i}$. If LL2 is applied, then the feasible time slot with the smallest length is first found and assigned to $t_{j,i}$. Note that the assigned time slot must be feasible for the task, i.e., the precedence constraint, the non-conflict constraint, and the deadline constraint must be met.

Each time a task is allocated, the allocated slot is deleted from the list, and new slots may be added to the list. There are four cases for slot assignment: (1) if the slot length is the same as the task processing time, the slot is removed directly; (2) if the start time of the task is the same as that of the slot, then the gap between the end time of the task and the end time of the slot will be added to the list as a new idle time slot; (3) if the end time of the task is the same as that of the slot, then the gap between the start time of the slot and the start time of the task forms a new idle slot; (4) otherwise, two gaps before and after the task are added to the list as new idle time slots. Every time a slot is assigned and removed, at most two new slots will be added. Therefore, there are at most $j+c-1$ nodes in the list when the $j^{th}$ task is assigned ($c$ is the initial number of nodes in the list). The time complexity for searching, inserting and removing a node on a list with $N$ nodes is $O(N)$. Allocating the total $n$ tasks to $VC_i$ takes the time complexity of $O(\sum_{j=1}^{n}(c+j-1)) = O(n^2)$. The time complexity for assigning all tasks is $O(mn^2)$.

### 4.2.3. Task prioritization

Given all ready tasks in $PQ$, two metrics are used to determine task priorities: task earliest start time and workflow instance priority. The earliest start time of $t_{j,i}$ can be calculated using Eq. (15). The priority of a workflow instance is determined by its position in the given workflow sequence.

$$\widetilde{b}_{j,i}^{*} = \max\{\widetilde{c}_{j,i'}^{*} + \widetilde{d}_{j,i',i} | t_{j,i'} \in Pt_{j,i}\} \tag{15}$$

Firstly, TP prioritizes ready tasks in $PQ$ according to their earliest fuzzy start time. The task with a smaller earliest start time has a higher priority. If two ready tasks have the same priority, then the second metric is used. The second metric is the priority of their workflow instances in the given $\pi$. A ready task inherits the priority of its workflow instance.

**Algorithm 3:** SAVNS ($\pi^0$, $\widetilde{F}^0$).

$\widetilde{PF}^* \leftarrow \{<\pi^0, \widetilde{F}^0>\}$;/* Non-denominated solution set */
$Ex \leftarrow \emptyset$;/* $Ex$ records explored sequences */
**while** $(\exists\pi) < \pi, \widetilde{F} > \in \widetilde{PF}^*$ *and* $\pi \notin Ex$ **do**
  Select an unexplored solution $<\pi, \widetilde{F}>$ from $\widetilde{PF}^*$;
  $T \leftarrow T_{ini}$;/* Start from an initial temperature */
  $k \leftarrow 1$; $flag = FALSE$;
  $\pi_{start} \leftarrow \pi$; $Ex \leftarrow Ex + \{\pi\}$;
  **while** $T < T_{fin}$ **do**
    **if** $k = 0$ **then**
      **break**;/* All neighborhood structures have been explored. */
    $Flag \leftarrow FALSE$; $Accept \leftarrow FALSE$;
    $\Pi_k \leftarrow \mathcal{N}_k(\pi_{start})$;/* Generate neighborhood by $N_k$ */
    **for** $\forall\pi' \in \Pi_k$ **do**
      $\widetilde{F}' \leftarrow FTS(\pi')$;
      **if** $\exists\widetilde{F}'' \in \widetilde{PF}^*$, $\widetilde{F}'' \prec \widetilde{F}'$ **then**
        Update $\widetilde{PF}^*$; $Flag \leftarrow TRUE$;
        $\pi_{start} \leftarrow \pi'$; Break;
      **else**
        /* Compute the crisp value by Eq. (7) */
        $C' \leftarrow C_1(\widetilde{C}')$; $T'_{tf} \leftarrow C_1(\widetilde{\mathcal{T}}'_{tf})$;
        $P \leftarrow \frac{e^{\frac{-C'}{kBxT}} + e^{\frac{-T'_{tf}}{kBxT}}}{2}$;
        $p \leftarrow Random(0,1)$;/* Random number between 0 and 1 */
        **if** $p < P$ **then**
          $\pi_{start} \leftarrow \pi'$; $Accept \leftarrow TRUE$; Break;
    **if** $Flag$ **then**
      $k \leftarrow 1$;
    **else**
      $k \leftarrow (k+1)\%log_2 n$;
    **if** $Accept$ **then**
      $T \leftarrow T \times \triangle t$;
**return** $\widetilde{PF}^*$

### 4.3. Simulated annealing and variable neighborhood search based heuristics

As a multi-objective problem, the obtained solutions should be evaluated by multiple metrics, i.e., the average degree of dissatisfaction and total rental cost. However, we cannot use the average degree of dissatisfaction to evaluate a fuzzy solution. A fuzzy solution always has zero degree of dissatisfaction, since it satisfies the deadline constraint. We can obtain the average degree of dissatisfaction only when the schedule becomes real. Therefore, we introduce two fuzzy metrics to evaluate the quality of a fuzzy schedule instead of our original objectives. One is the fuzzy total rental cost $\widetilde{C}$ which can be computed by Eqs. (7)–(11). The other is the fuzzy total flow time $\widetilde{\mathcal{T}}_{tf}$. The total fuzzy flow time $\widetilde{\mathcal{T}}_{tf}$ is the sum of the fuzzy completion times of all workflow instances. It can be obtained by Eq. (16). A smaller $\widetilde{\mathcal{T}}_{tf}$ means a shorter average completion time of workflow instances, which may lead to a smaller average degree of dissatisfaction.

$$\widetilde{\mathcal{T}}_{tf} = \sum_{j=1}^{n} \widetilde{c}_{j,m+1} \tag{16}$$

It is possible to find better solutions for the problem considered by adjusting the order of tasks to be arranged. Since tasks in a workflow are constrained by the precedence constraints, new task sequences created by simply swapping some pairs of tasks could result in infeasible solutions. In the task prioritization component, the workflow sequence plays a key role at the beginning to prioritize the first batch of ready tasks. The first batch of ready tasks are those whose predecessor task is

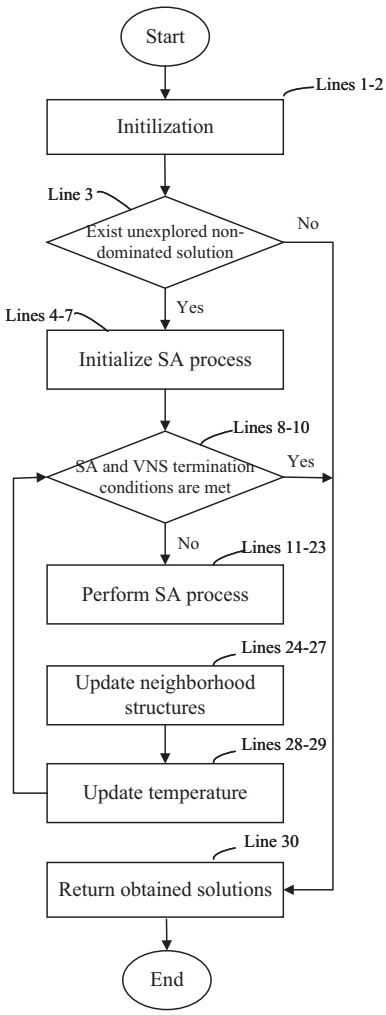



**Fig. 5.** Flowchart of SAVNS.

**Table 3**
Parameter settings.

| Param. | Value Setting | Param. | Value Setting |
|---|---|---|---|
| $n$ | 25, 50, 75, 100 | $m$ | 5, 6, 7, 8, 9, 10 |
| $e$ | 0.1, 0.2, 0.3 | $os$ | 0.2, 0.4, 0.6, 0.8 |
| $p^2_{j,i}$ | $U(1000, 10000)$ ms | $p^1_{j,i}$ | $p^2_{j,i} \times (1-e)$ |
| $p^3_{j,i}$ | $p^2_{j,i} \times (1+e)$ | $p_{i,j}$ | $N(p^2_{j,i}, (e \times p^2_{j,i})^2)$ |
| $d^2_{j,i}$ | $U(250, 280)$ ms | $d^1_{j,i}$ | $d^2_{j,i} \times (1-e)$ |
| $d^3_{j,i}$ | $d^2_{j,i} \times (1+e)$ | $d_{i,j}$ | $N(d^2_{j,i}, (e \times d^2_{j,i})^2)$ |
| $df_{avg}$ | 1.5, 2, 2.5, 3 | $df_j$ | $N(df_{avg}, (0.1 \times df_{avg})^2)$ |
| $D^2_j$ | $df_j \times c^2_{j,m+1}$ | $D^3_j$ | $df_j \times c^3_{j,m+1}$ |
| $u$ | 600 s | $\|R_i\|$ | $n/(df_{avg} \times os)$ |
| $\psi^r_i$ | 0.25 | $\psi^o_i$ | 1 |

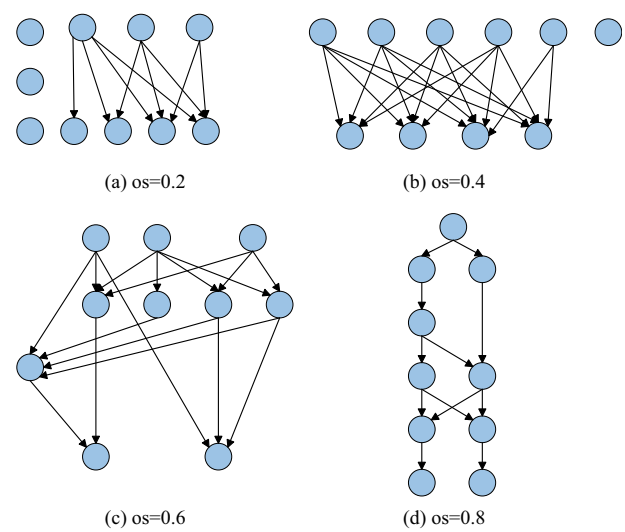



**Fig. 6.** Examples of DAG with different hardness.

the dummy source. Their priorities are mainly determined by the workflow sequence. In this paper, the SAVNS method is used to improve the solution. The main framework of SAVNS is shown in Algorithm 3. The flowchart of SAVNS is illustrated in Fig. 5.

The set $\widetilde{PF}^*$ is used to restore all non-dominated fuzzy solutions. Initially, it has only one solution, i.e., the initial solution $< \pi^0, \widetilde{F}^0 >$ (Line 1). New non-dominated solutions are added to $\widetilde{PF}^*$ during the improvement process of SAVNS. In order to avoid redundant computation, the set $Ex$ is used to keep explored sequences (Line 2). In each iteration, a solution in $\widetilde{PF}^*$ but not in $Ex$ is randomly selected (Line 4) to explore (Lines 8–29).

The solution exploring process conducts an SA-based local search procedure with varied neighborhood structures. The local search stops when the system is frozen (i.e., the temperature reaches the final temperature). In each iteration of the improvement process, three main steps are performed.

Firstly, the neighborhood $\Pi_k$ is constructed (Line 12). $\mathcal{N}_k$ ($k = 1, 2, \ldots, log_2\ n$) is a neighborhood structure that swaps two workflows with a specified swapping distance $\frac{n}{2^k}$, i.e., a new sequence is obtained by swapping $\pi[j]$ and $\pi[(j + \frac{n}{2^k})\%n]$. The variable neighborhood structures are generated by decreasing the cardinality of the swapping distance. For example, if $\pi = (W_1, W_4, W_3, W_2)$, then it has only two neighborhood structures $\mathcal{N}_1$ and $\mathcal{N}_2$ as below. The swapping distance of $\mathcal{N}_1$ is 2 and the swapping distance of $\mathcal{N}_2$ is 1.

$$\mathcal{N}_1 = \{((W_3, W_4, W_1, W_2), (W_1, W_2, W_3, W_4)\}$$
$$\mathcal{N}_2 = \{(W_4, W_1, W_3, W_2), (W_1, W_3, W_4, W_2), (W_1, W_4, W_2, W_3), (W_2, W_4, W_3, W_1)\}$$

Secondly, the fuzzy solutions of neighbors in $\mathcal{N}_k$ are computed by calling Fuzzy Transition System (FTS) (Line 14). The solutions obtained are evaluated and $\widetilde{PF}^*$ is updated (Lines 15–17). A dominated solution has a probability of being accepted for the next iteration (Lines 19–23).

Thirdly, the neighborhood structure is changed if no non-dominated solution is found in the iteration (Lines 24–27). If all neighborhood structures have been explored (Line 9) but no improvement is made, the iteration stops. The temperature is updated if a dominated solution is accepted for the next iteration (Lines 28–29).

In each iteration of solution improving process (Lines 8–29), at most $n$ neighbors are explored by FTS. Therefore, the worst time complexity of each iteration is $O(mn^3)$. The time complexity of SAVNS cannot be expressed in a closed form, since the number of iterations is difficult to determine.

## 5. Computational experiments

In this section, we first introduce the metrics to evaluate algorithm performance and the instance parameter settings. Then we calibrate some key components of our proposal. Finally, we compare the calibrated algorithm with several baseline algorithms on the aspects of effectiveness, efficiency and robustness. All algorithms are coded in Java and run on a PC with an Intel Core i7-4790 3.6 GHz processor and 8GB of RAM. All experiments are performed in the simulated clusters. The

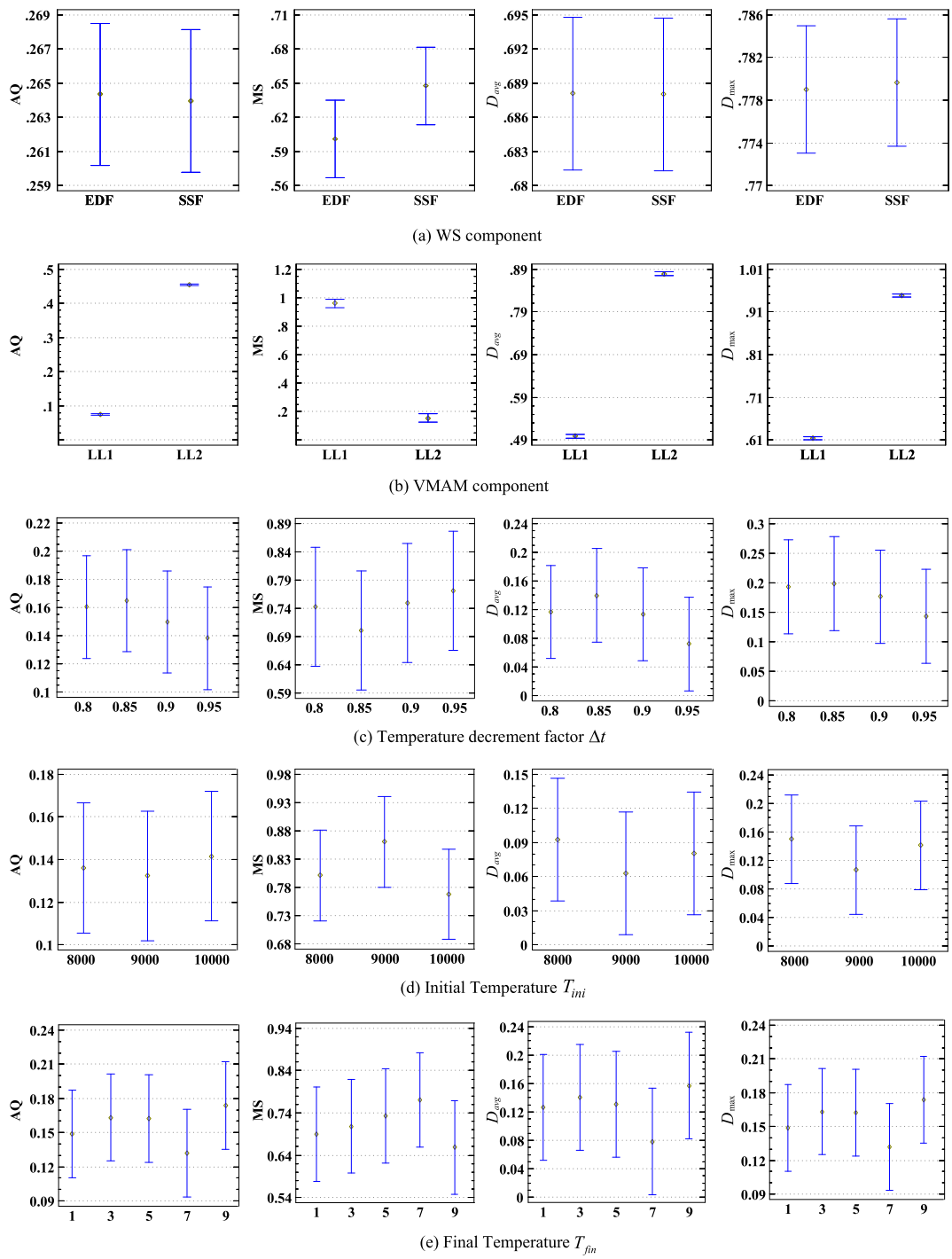



**Fig. 7.** Pareto metrics for algorithm variants with different component and parameter settings with 95 % Tukey HSD intervals.

testing instances and the codes of all algorithms are available on GitHub (https://github.com/Fengmei-Liu/FuzzySchedule.git).

### 5.1. Parameter setting

We generate testing instances based on studies in [5,27] that investigate the related problem considered. Four Pareto-based metrics [28] are adopted:

- The average quality ($AQ$) metric measures the average quality of the set of solutions. The lower $AQ$ metric indicates the better performance.
- The maximum spread ($MS$) metric measures how well the Pareto front is covered by the set of obtained solutions. The optimal Pareto front is difficult to obtain, since the problem considered is NP-hard. The approximate non-dominated set which contains the non-dominated solutions obtained by all compared algorithms is used as the near-optimal Pareto front. A larger $MS$ implies that more of the Pareto front is covered by the set of obtained solutions.
- The average distance ($D_{avg}$) metric is defined as the average distance between Pareto front and the set of obtained solutions. A smaller $D_{avg}$ means a better approximation to the Pareto front.

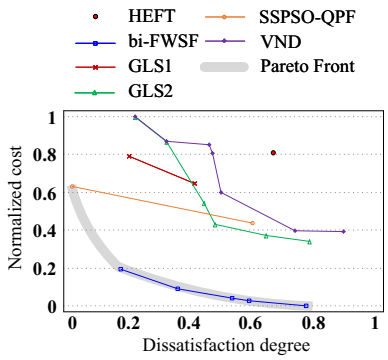



**Fig. 8.** Non-dominated solutions for a CyberShake instance.

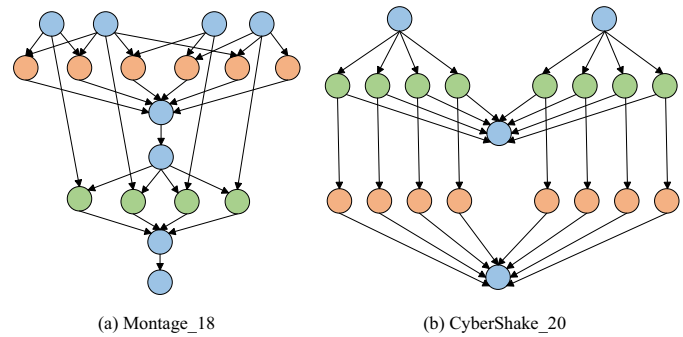



**Fig. 10.** Workflows.

- The maximum distance ($D_{max}$) metric measures the maximum distance between the Pareto front and the set of obtained solutions. Similarly to $D_{avg}$, the smaller $D_{max}$, the better.

Table 3 summarizes the main parameters for the testing instances. A random network generator *RanGen* [29] is used to randomly generate the DAG structure of a testing instance. *RanGen* defines a factor $os$ to control the hardness of the DAG generated. A smaller $os$ means a higher concurrency of the workflow structure. Fig. 6 shows examples of DAG with different concurrency. Note that the number of reserved VMs in $R_i$ is closely related to $n$, $df_{avg}$ and $os$. A higher $df_{avg}$ indicates the less urgency of the workflows. Therefore, $|R_i|$ is set to be proportional to $n$ and inversely proportional to $df_{avg}$ and $os$. The pricing models of VMs are set as in the Amazon Elastic Compute Cloud.

### 5.2. Component calibration

There are $4 \times 6 \times 3 \times 4 \times 4 = 1152$ instance combinations for calibration ($n \in \{25, 50, 75, 100\}$, $m \in \{5, 6, 7, 8, 9, 10\}$, $e \in \{0.1, 0.2, 0.3\}$, $df_{avg} \in \{1.5, 2, 2.5, 3\}$, and $os \in \{0.2, 0.4, 0.6, 0.8\}$). We randomly generate 1 testing instance for each possible combination. We calibrate the components of the proposed algorithm on $1152 \times 1 = 1152$ instances in total. The performance of the component variants is evaluated by $AQ$, $MS$, $D_{avg}$ and $D_{max}$. In the proposed bi-FWSF, there are two variants for the VMAM (LL1 and LL2), two variants for the workflow sequencing methods (EDDF and SSTF), three parameter settings for the initial temperature ($T_{ini} \in \{5000, 10000, 15000\}$), three parameter settings for the final temperature ($T_{fin} \in \{1, 5, 10\}$) and four parameter settings for the temperature decrement factor $\Delta t \in \{0.8, 0.85, 0.9, 0.95\}$. Therefore, there are $2 \times 2 \times 3 \times 3 \times 4 = 144$ combinations of components and parameters for the proposal. In total, $1152 \times 144 = 165,888$ experimental results are obtained for the calibration of the components.

Fig. 7 (a) shows the four Pareto metrics for algorithm variants with different WS components. The performance of EDDF and SSTF does not differ significantly for the metrics $AQ$, $D_{avg}$ and $D_{max}$. This means that EDDF and SSTF have a similar effect on the solution quality and approximation to the Pareto Front. However, SSTF has a relatively better performance on $MS$ than EDDF, which indicates that SSTF could lead to better coverage on the Pareto Front than EDDF.

Fig. 7 (c)–(e) show the four Pareto metrics for algorithm variants with different SA parameters. Algorithm variants with different parameter settings are not significantly different on the Pareto metrics. However, the variants with $\Delta t = 0.95$, $T_{ini} = 9000$ and $T_{fin} = 7$ have the best average performance.

Based on the results of the calibration experiments, we set SSTF for workflow sequencing, LL1 for VMAM, $\Delta t = 0.95$, $T_{ini} = 9000$ and $T_{fin} = 7$ for the proposed bi-FWSF.

### 5.3. Algorithm comparison

The proposed bi-FWSF is compared with five baseline algorithms: improved Heterogeneous Earliest Finish Time (HEFT) [30], VND [3], Greedy Local Search methods (GLS1 and GLS2) [31] and SSPSO-QPF [26]. HEFT, VND, GLS1 and GLS2 are not developed for fuzzy scheduling. Therefore, we modify them to operate on fuzzy temporal parameters. In addition, VND in [3] is used to schedule the jobs arriving in one time period (1 s). It is restricted to running in one second to output the solution on time. Since in the paper the problem is static, we remove the restriction. GLS1 and GLS2 are the two variants of greedy local search methods with different neighborhood structures. GLS1 employs one insertion and GLS2 adapts pairwise switching as the neighborhood structure. SSPSO-QPF is developed for the dynamic and fuzzy workflow scheduling problem where $n$ ($n = 5, 10, 20, 30, 40, 50$) workflows arrive at the system at a given arrival rate. The algorithm can be modified to solve the problem considered by allowing all workflows to arrive at the same time.

Firstly, one specific instance based on a real-world workflow CyberShake is produced with $m = 20, n = 25, e = 0.2, os = 0.2, df_{avg} = 1.5$. The scatter plots of solutions obtained by all compared algorithms for the instance are shown in Fig. 8. The proposed bi-FWSF obtains the best set of solutions for the instance since it contributes five out of six

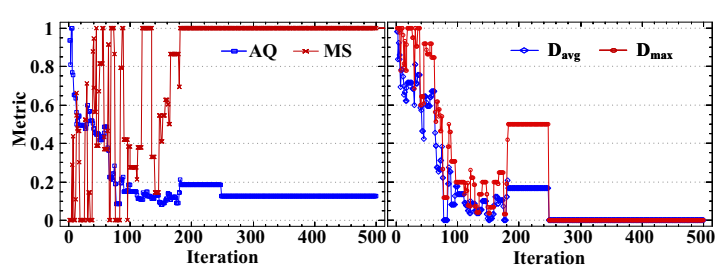



**Fig. 9.** Non-dominated solutions for a CyberShake instance.

**Table 4**
Experimental results on real-world workflow instances.

| Param. | Value | $AQ$ | | | | | | $MS$ | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | HEFT | GLS1 | GLS2 | VND | SSPSO-QPF | bi-FWSF | HEFT | GLS1 | GLS2 | VND | SSPSO-QPF | bi-FWSF |
| $m$ | 18 | 0.651 | 0.328 | 0.138 | 0.149 | 0.331 | **0.118** | 0.000 | 0.086 | 0.520 | 0.516 | 0.089 | **0.749** |
| | 20 | 0.710 | 0.301 | 0.179 | 0.173 | 0.285 | **0.132** | 0.000 | 0.097 | 0.476 | 0.494 | 0.218 | **0.732** |
| $n$ | 25 | 0.660 | 0.272 | 0.197 | 0.187 | 0.244 | **0.134** | 0.000 | 0.085 | 0.436 | 0.485 | 0.367 | **0.736** |
| | 50 | 0.694 | 0.295 | 0.126 | 0.125 | 0.298 | **0.114** | 0.000 | 0.101 | 0.621 | 0.643 | 0.106 | **0.789** |
| | 75 | 0.687 | 0.351 | 0.148 | 0.154 | 0.338 | **0.124** | 0.000 | 0.084 | 0.643 | 0.643 | 0.041 | **0.790** |
| | 100 | 0.682 | 0.340 | 0.164 | 0.180 | 0.353 | **0.127** | 0.000 | 0.095 | 0.292 | 0.250 | 0.099 | **0.647** |
| $e$ | 0.1 | 0.678 | 0.344 | 0.174 | 0.184 | 0.315 | **0.108** | 0.000 | 0.051 | 0.375 | 0.395 | 0.110 | **0.736** |
| | 0.2 | 0.688 | 0.287 | 0.140 | 0.141 | 0.317 | **0.127** | 0.000 | 0.125 | 0.572 | 0.549 | 0.163 | **0.723** |
| | 0.3 | 0.675 | 0.312 | 0.163 | 0.159 | 0.292 | **0.140** | 0.000 | 0.098 | 0.547 | 0.572 | 0.187 | **0.763** |
| $df_{avg}$ | 1.5 | 0.685 | 0.279 | 0.150 | 0.149 | 0.272 | **0.117** | 0.000 | 0.098 | 0.474 | 0.476 | 0.181 | **0.819** |
| | 2 | 0.660 | 0.302 | 0.185 | 0.202 | 0.291 | **0.134** | 0.000 | 0.107 | 0.513 | 0.497 | 0.152 | **0.756** |
| | 2.5 | 0.685 | 0.329 | 0.177 | 0.172 | 0.332 | **0.142** | 0.000 | 0.068 | 0.450 | 0.479 | 0.179 | **0.629** |
| | 3 | 0.692 | 0.347 | 0.123 | 0.123 | 0.338 | **0.106** | 0.000 | 0.092 | 0.556 | 0.570 | 0.101 | **0.759** |
| Average | | 0.680 | 0.314 | 0.159 | 0.161 | 0.308 | **0.125** | 0.000 | 0.091 | 0.498 | 0.505 | 0.153 | **0.741** |

**Table 5**
Experimental results on real-world workflow instances.

| Param. | Value | $D_{avg}$ | | | | | | $D_{max}$ | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | HEFT | GLS1 | GLS2 | VND | SSPSO-QPF | bi-FWSF | HEFT | GLS1 | GLS2 | VND | SSPSO-QPF | bi-FWSF |
| $m$ | 18 | 0.865 | 0.414 | 0.123 | 0.135 | 0.419 | **0.053** | 0.995 | 0.595 | 0.264 | 0.281 | 0.627 | **0.180** |
| | 20 | 0.890 | 0.363 | 0.137 | 0.129 | 0.318 | **0.065** | 1.000 | 0.585 | 0.281 | 0.289 | 0.527 | **0.187** |
| $n$ | 25 | 0.862 | 0.306 | 0.173 | 0.155 | 0.255 | **0.087** | 0.999 | 0.527 | 0.337 | 0.353 | 0.512 | **0.222** |
| | 50 | 0.894 | 0.365 | 0.091 | 0.087 | 0.365 | **0.046** | 1.000 | 0.528 | 0.197 | 0.190 | 0.549 | **0.142** |
| | 75 | 0.880 | 0.449 | 0.105 | 0.110 | 0.425 | **0.034** | 1.000 | 0.656 | 0.260 | 0.266 | 0.615 | **0.143** |
| | 100 | 0.873 | 0.435 | 0.151 | 0.177 | 0.428 | **0.069** | 0.991 | 0.649 | 0.295 | 0.330 | 0.630 | **0.227** |
| $e$ | 0.1 | 0.866 | 0.464 | 0.202 | 0.218 | 0.388 | **0.064** | 0.995 | 0.660 | 0.353 | 0.394 | 0.590 | **0.203** |
| | 0.2 | 0.902 | 0.331 | 0.085 | 0.085 | 0.372 | **0.054** | 1.000 | 0.524 | 0.227 | 0.230 | 0.586 | **0.171** |
| | 0.3 | 0.865 | 0.371 | 0.104 | 0.093 | 0.345 | **0.058** | 0.997 | 0.586 | 0.237 | 0.230 | 0.554 | **0.176** |
| $df_{avg}$ | 1.5 | 0.865 | 0.315 | 0.123 | 0.119 | 0.315 | **0.038** | 0.996 | 0.520 | 0.288 | 0.286 | 0.507 | **0.135** |
| | 2 | 0.867 | 0.356 | 0.149 | 0.176 | 0.337 | **0.053** | 0.995 | 0.573 | 0.291 | 0.331 | 0.553 | **0.185** |
| | 2.5 | 0.870 | 0.422 | 0.149 | 0.141 | 0.386 | **0.092** | 0.999 | 0.589 | 0.319 | 0.332 | 0.572 | **0.264** |
| | 3 | 0.908 | 0.462 | 0.100 | 0.092 | 0.436 | **0.052** | 1.000 | 0.678 | 0.191 | 0.190 | 0.675 | **0.151** |
| Average | | 0.878 | 0.389 | 0.130 | 0.132 | 0.368 | **0.059** | 0.997 | 0.590 | 0.272 | 0.285 | 0.577 | **0.183** |

**Table 6**
Experimental results on simulated instances.

| Param. | Value | $AQ$ | | | | | | $MS$ | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | HEFT | GLS1 | GLS2 | VND | SSPSO-QPF | bi-FWSF | HEFT | GLS1 | GLS2 | VND | SSPSO-QPF | bi-FWSF |
| $m$ | 5 | 0.575 | 0.227 | 0.068 | 0.091 | 0.185 | **0.055** | 0.000 | 0.024 | 0.290 | 0.214 | 0.012 | **0.433** |
| | 6 | 0.569 | 0.230 | 0.068 | 0.092 | 0.177 | **0.052** | 0.000 | 0.015 | 0.282 | 0.201 | 0.031 | **0.477** |
| | 7 | 0.531 | 0.204 | 0.073 | 0.089 | 0.173 | **0.059** | 0.000 | 0.032 | 0.292 | 0.228 | 0.021 | **0.443** |
| | 8 | 0.549 | 0.203 | 0.070 | 0.093 | 0.183 | **0.056** | 0.000 | 0.024 | 0.300 | 0.211 | 0.016 | **0.472** |
| | 9 | 0.543 | 0.211 | 0.074 | 0.093 | 0.226 | **0.058** | 0.000 | 0.017 | 0.279 | 0.221 | 0.018 | **0.458** |
| | 10 | 0.514 | 0.166 | 0.075 | 0.075 | 0.192 | **0.065** | 0.000 | 0.027 | 0.278 | 0.282 | 0.026 | **0.388** |
| $n$ | 25 | 0.527 | 0.133 | 0.075 | **0.053** | 0.109 | 0.061 | 0.000 | 0.034 | 0.186 | **0.346** | 0.045 | 0.281 |
| | 50 | 0.550 | 0.179 | 0.073 | **0.058** | 0.190 | 0.066 | 0.000 | 0.040 | 0.286 | **0.377** | 0.019 | 0.343 |
| | 75 | 0.561 | 0.243 | 0.061 | 0.107 | 0.202 | **0.052** | 0.000 | 0.013 | 0.440 | 0.101 | 0.011 | **0.567** |
| | 100 | 0.558 | 0.294 | 0.077 | 0.149 | 0.256 | **0.047** | 0.000 | 0.003 | 0.238 | 0.041 | 0.008 | **0.633** |
| $e$ | 0.1 | 0.536 | 0.201 | 0.062 | 0.083 | 0.182 | **0.047** | 0.000 | 0.022 | 0.290 | 0.226 | 0.022 | **0.477** |
| | 0.2 | 0.552 | 0.210 | 0.073 | 0.092 | 0.192 | **0.059** | 0.000 | 0.024 | 0.290 | 0.220 | 0.018 | **0.447** |
| | 0.3 | 0.556 | 0.215 | 0.078 | 0.093 | 0.194 | **0.065** | 0.000 | 0.023 | 0.282 | 0.225 | 0.035 | **0.422** |
| $os$ | 0.2 | 0.516 | 0.228 | 0.088 | 0.099 | 0.186 | **0.077** | 0.000 | 0.014 | 0.289 | 0.232 | 0.040 | **0.444** |
| | 0.4 | 0.543 | 0.223 | 0.090 | 0.098 | 0.184 | **0.075** | 0.000 | 0.016 | 0.240 | 0.233 | 0.034 | **0.408** |
| | 0.6 | 0.586 | 0.216 | 0.061 | 0.079 | 0.172 | **0.045** | 0.000 | 0.008 | 0.278 | 0.214 | 0.010 | **0.453** |
| | 0.8 | 0.548 | 0.168 | 0.045 | 0.081 | 0.136 | **0.031** | 0.000 | 0.055 | 0.343 | 0.215 | 0.014 | **0.487** |
| $df_{avg}$ | 1.5 | 0.598 | 0.245 | 0.088 | 0.121 | 0.196 | **0.070** | 0.000 | 0.009 | 0.247 | 0.136 | 0.033 | **0.403** |
| | 2 | 0.563 | 0.214 | 0.073 | 0.092 | 0.189 | **0.059** | 0.000 | 0.021 | 0.280 | 0.221 | 0.029 | **0.444** |
| | 2.5 | 0.529 | 0.196 | 0.065 | 0.078 | 0.191 | **0.052** | 0.000 | 0.032 | 0.293 | 0.259 | 0.018 | **0.455** |
| | 3 | 0.503 | 0.182 | 0.058 | 0.068 | 0.181 | **0.047** | 0.000 | 0.030 | 0.330 | 0.277 | 0.019 | **0.489** |
| Average | | 0.548 | 0.209 | 0.071 | 0.090 | 0.185 | **0.057** | 0.000 | 0.023 | 0.287 | 0.223 | 0.023 | **0.448** |

**Table 7**
Experimental results on simulated instances.

| Param. | Value | $D_{avg}$ | | | | | | $D_{max}$ | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | HEFT | GLS1 | GLS2 | VND | SSPSO-QPF | bi-FWSF | HEFT | GLS1 | GLS2 | VND | SSPSO-QPF | bi-FWSF |
| $m$ | 5 | 0.960 | 0.748 | 0.387 | 0.460 | 0.634 | **0.288** | 0.999 | 0.834 | 0.499 | 0.556 | 0.725 | **0.390** |
| | 6 | 0.964 | 0.757 | 0.391 | 0.458 | 0.582 | **0.265** | 0.999 | 0.840 | 0.506 | 0.559 | 0.709 | **0.370** |
| | 7 | 0.953 | 0.705 | 0.367 | 0.435 | 0.648 | **0.266** | 0.998 | 0.803 | 0.483 | 0.542 | 0.752 | **0.373** |
| | 8 | 0.952 | 0.695 | 0.349 | 0.432 | 0.689 | **0.242** | 0.998 | 0.796 | 0.474 | 0.548 | 0.789 | **0.358** |
| | 9 | 0.948 | 0.689 | 0.336 | 0.398 | 0.630 | **0.225** | 0.996 | 0.796 | 0.474 | 0.524 | 0.750 | **0.357** |
| | 10 | 0.938 | 0.623 | 0.337 | 0.329 | 0.613 | **0.266** | 0.995 | 0.743 | 0.471 | 0.464 | 0.734 | **0.399** |
| $n$ | 25 | 0.955 | 0.650 | 0.458 | **0.325** | 0.551 | 0.380 | 0.998 | 0.750 | 0.580 | **0.439** | 0.676 | 0.499 |
| | 50 | 0.949 | 0.661 | 0.342 | **0.288** | 0.604 | 0.306 | 0.998 | 0.766 | 0.468 | **0.407** | 0.717 | 0.431 |
| | 75 | 0.952 | 0.729 | 0.262 | 0.506 | 0.673 | **0.181** | 0.997 | 0.829 | 0.381 | 0.618 | 0.781 | **0.292** |
| | 100 | 0.956 | 0.801 | 0.385 | 0.601 | 0.702 | **0.145** | 0.998 | 0.887 | 0.508 | 0.705 | 0.800 | **0.249** |
| $e$ | 0.1 | 0.963 | 0.755 | 0.406 | 0.464 | 0.672 | **0.270** | 0.999 | 0.837 | 0.511 | 0.562 | 0.772 | **0.365** |
| | 0.2 | 0.949 | 0.692 | 0.345 | 0.414 | 0.630 | **0.252** | 0.998 | 0.795 | 0.471 | 0.527 | 0.744 | **0.370** |
| | 0.3 | 0.948 | 0.676 | 0.340 | 0.395 | 0.595 | **0.254** | 0.997 | 0.786 | 0.475 | 0.521 | 0.714 | **0.384** |
| $os$ | 0.2 | 0.908 | 0.566 | 0.205 | 0.240 | 0.388 | **0.145** | 0.995 | 0.717 | 0.367 | 0.390 | 0.556 | **0.298** |
| | 0.4 | 0.930 | 0.597 | 0.267 | 0.283 | 0.447 | **0.180** | 0.996 | 0.739 | 0.431 | 0.435 | 0.613 | **0.332** |
| | 0.6 | 0.977 | 0.773 | 0.399 | 0.464 | 0.766 | **0.268** | 1.000 | 0.852 | 0.525 | 0.580 | 0.857 | **0.386** |
| | 0.8 | 0.997 | 0.891 | 0.578 | 0.708 | 0.929 | **0.439** | 1.000 | 0.910 | 0.617 | 0.738 | 0.947 | **0.477** |
| $df_{avg}$ | 1.5 | 0.942 | 0.660 | 0.328 | 0.418 | 0.564 | **0.228** | 0.999 | 0.780 | 0.472 | 0.549 | 0.693 | **0.367** |
| | 2 | 0.953 | 0.706 | 0.366 | 0.422 | 0.633 | **0.264** | 0.998 | 0.806 | 0.489 | 0.537 | 0.744 | **0.378** |
| | 2.5 | 0.957 | 0.723 | 0.381 | 0.423 | 0.664 | **0.274** | 0.997 | 0.813 | 0.499 | 0.530 | 0.766 | **0.384** |
| | 3 | 0.961 | 0.739 | 0.375 | 0.430 | 0.669 | **0.267** | 0.997 | 0.820 | 0.481 | 0.528 | 0.770 | **0.364** |
| Average | | 0.953 | 0.707 | 0.362 | 0.423 | 0.632 | **0.258** | 0.998 | 0.805 | 0.485 | 0.536 | 0.743 | **0.373** |

solutions to the Pareto Front. The convergence behavior of bi-FWSF on four metrics is illustrated in Fig. 9. The solutions obtained during the iterations are fuzzy and cannot be evaluated by Pareto metrics. We turn these fuzzy solutions into real ones and then evaluate them afterwards. It is possible that a good fuzzy solution leads to a real solution of low quality, or a not so good fuzzy solution that may result in a good real solution is eliminated in successive iterations. These bring about the unstable convergence of bi-FWSF. However, the convergence curve of bi-FWSF becomes stable after 200 iterations.

Secondly, the compared algorithms are tested on many instances based on two real-world workflows: Montage_18 and CyberShake_20 (Fig. 10). There are $4 \times 2 \times 3 \times 4 = 96$ instances based on real-world workflows ($n \in \{25, 50, 75, 100\}$, $m \in \{18, 20\}$, $e \in \{0.1, 0.2, 0.3\}$ and $df_{avg} \in \{1.5, 2, 2.5, 3\}$).

The experimental results on real-world workflow instances are shown in Tables 4–5. We can find out that bi-FWSF obtains the best results on all four metrics.

Thirdly, the compared algorithms are tested on simulated instances. There are 1152 combinations for simulated instances ($n \in \{25, 50, 75, 100\}$, $m \in \{5, 6, 7, 8, 9, 10\}$, $e \in \{0.1, 0.2, 0.3\}$, $df_{avg} \in \{1.5, 2, 2.5, 3\}$, and $os \in \{0.2, 0.4, 0.6, 0.8\}$). We randomly generate 1 new testing instance for each combination.

The experimental results on the simulated instances are given in Tables 6–8. Table 6 shows that bi-FWSF obtains the best $AQ$ with different parameter settings, except for $n = 25, 50$. bi-FWSF has the lowest average $AQ$ (0.057), GLS2 has the second lowest (0.071), while HEFT has the highest average $AQ$ (0.548). bi-FWSF achieves the highest average $MS$ (0.448), GLS2 has the second highest (0.287), and HEFT has the worst (0.000).

Similarly, Table 7 shows that bi-FWSF has the best performance on both $D_{avg}$ (0.258) and $D_{max}$ (0.373) in most cases, except for $n = 25, 50$. GLS2 is the second best algorithm on both distance metrics. HEFT is the worst.

Table 8 illustrates the computation time of compared algorithms. It shows that HEFT is the fastest algorithm. It takes no more than one second for all testing instances. It is not difficult to understand the reason. HEFT generates only one solution. The average computation time of bi-FWSF is about 3–5 times of the other local search algorithms (GLS1, GLS2 and VND) and much more than that of HEFT. bi-FWSF is the slowest one. bi-FWSF employs the simulated annealing process to force the algorithm to jump out of the local optima. In addition, variable neighborhood structures with decreased cardinality of the swapping distance can significantly extend the search time. Through a slow convergence process, the algorithm finds better solution.

Fig. 11 shows the interactions between the instance parameters ($m$, $n$, $e$, $os$, $df_{avg}$) and the algorithms compared. We can observe whether the performance differences of these algorithms are significant or not. The sensitivity of these algorithms to the changes in instance parameters can also be observed.

For instances with different stage numbers $m$ (see the first column of the figure), bi-FWSF outperforms all the other algorithms significantly on the all four metrics. bi-FWSF is not sensitive to changes in the parameter $m$.

For instances with different workflow numbers $n$ (see the second column of the figure), bi-FWSF outperforms the other algorithms significantly on $AQ$ metric. It is also not sensitive to changes in the parameter $n$ for the $AQ$ metric. For the other metrics, bi-FWSF outperforms the other algorithms significantly when $n = 75$ and $n = 100$. It is sensitive to changes in $n$ for the other three metrics.

For instances with different uncertainty levels (see the third column of the figure), bi-FWSF outperforms the other algorithms significantly on all metrics. It is also the most stable algorithm among all compared ones.

For instances with different DAG hardness of workflow (see the fourth column of the figure), bi-FWSF outperforms the other algorithms significantly on all metrics. But it is sensitive to the parameter changes in the distance metrics.

For instances with different DAG hardness of workflows (see the last column of the figure), bi-FWSF outperforms the other algorithms significantly on all metrics. It is also the most robust algorithm among all the compared ones.

Based on the results of the compared experiments, we find that bi-FWSF is the most effective and robust method compared to all the other baseline algorithms. The computation time needed by bi-FWSF is about 3–5 times that of the compared local search algorithms and much more than that of HEFT. However, considering the high-quality solutions

**Table 8**
Computation time of compared algorithms.

| $m \times n$ | Time (s) | | | | | |
|---|---|---|---|---|---|---|
| | HEFT | GLS1 | GLS2 | VND | SSPSO-QPF | bi-FWSF |
| 5 × 25 | 0.01 | 1.21 | 0.73 | 6.06 | 15.09 | 1.80 |
| 5 × 50 | 0.01 | 32.53 | 17.62 | 118.97 | 46.71 | 32.54 |
| 5 × 75 | 0.01 | 105.25 | 91.30 | 120.41 | 64.74 | 224.10 |
| 5 × 100 | 0.01 | 119.79 | 118.07 | 120.76 | 68.09 | 867.61 |
| 6 × 25 | 0.01 | 1.37 | 0.83 | 6.95 | 24.05 | 2.08 |
| 6 × 50 | 0.01 | 35.62 | 18.47 | 118.55 | 50.71 | 35.84 |
| 6 × 75 | 0.01 | 109.39 | 91.14 | 120.45 | 70.08 | 227.01 |
| 6 × 100 | 0.02 | 120.15 | 118.76 | 120.86 | 71.69 | 820.87 |
| 7 × 25 | 0.01 | 1.53 | 1.00 | 7.21 | 31.66 | 2.45 |
| 7 × 50 | 0.01 | 39.62 | 20.20 | 117.85 | 77.80 | 38.19 |
| 7 × 75 | 0.01 | 107.66 | 84.90 | 120.54 | 93.87 | 226.84 |
| 7 × 100 | 0.02 | 120.72 | 116.82 | 121.04 | 94.92 | 786.22 |
| 8 × 25 | 0.01 | 1.56 | 0.95 | 7.55 | 43.21 | 2.52 |
| 8 × 50 | 0.01 | 39.18 | 19.55 | 116.74 | 104.35 | 39.37 |
| 8 × 75 | 0.01 | 107.11 | 84.16 | 120.61 | 111.94 | 225.69 |
| 8 × 100 | 0.02 | 121.14 | 115.91 | 121.15 | 113.28 | 806.84 |
| 9 × 25 | 0.01 | 1.74 | 1.02 | 7.85 | 40.68 | 2.76 |
| 9 × 50 | 0.01 | 39.94 | 16.99 | 116.02 | 87.70 | 36.86 |
| 9 × 75 | 0.02 | 105.48 | 79.85 | 120.69 | 113.88 | 218.24 |
| 9 × 100 | 0.02 | 119.64 | 114.33 | 121.37 | 115.63 | 780.54 |
| 10 × 25 | 0.01 | 1.81 | 1.08 | 8.07 | 48.54 | 2.95 |
| 10 × 50 | 0.01 | 40.36 | 19.23 | 116.48 | 123.50 | 41.45 |
| 10 × 75 | 0.02 | 108.59 | 79.59 | 120.80 | 184.57 | 221.98 |
| 10 × 100 | 0.03 | 125.75 | 120.29 | 122.98 | 194.66 | 800.34 |
| Average | 0.014 | 66.963 | 55.533 | 91.664 | 80.13 | 268.546 |

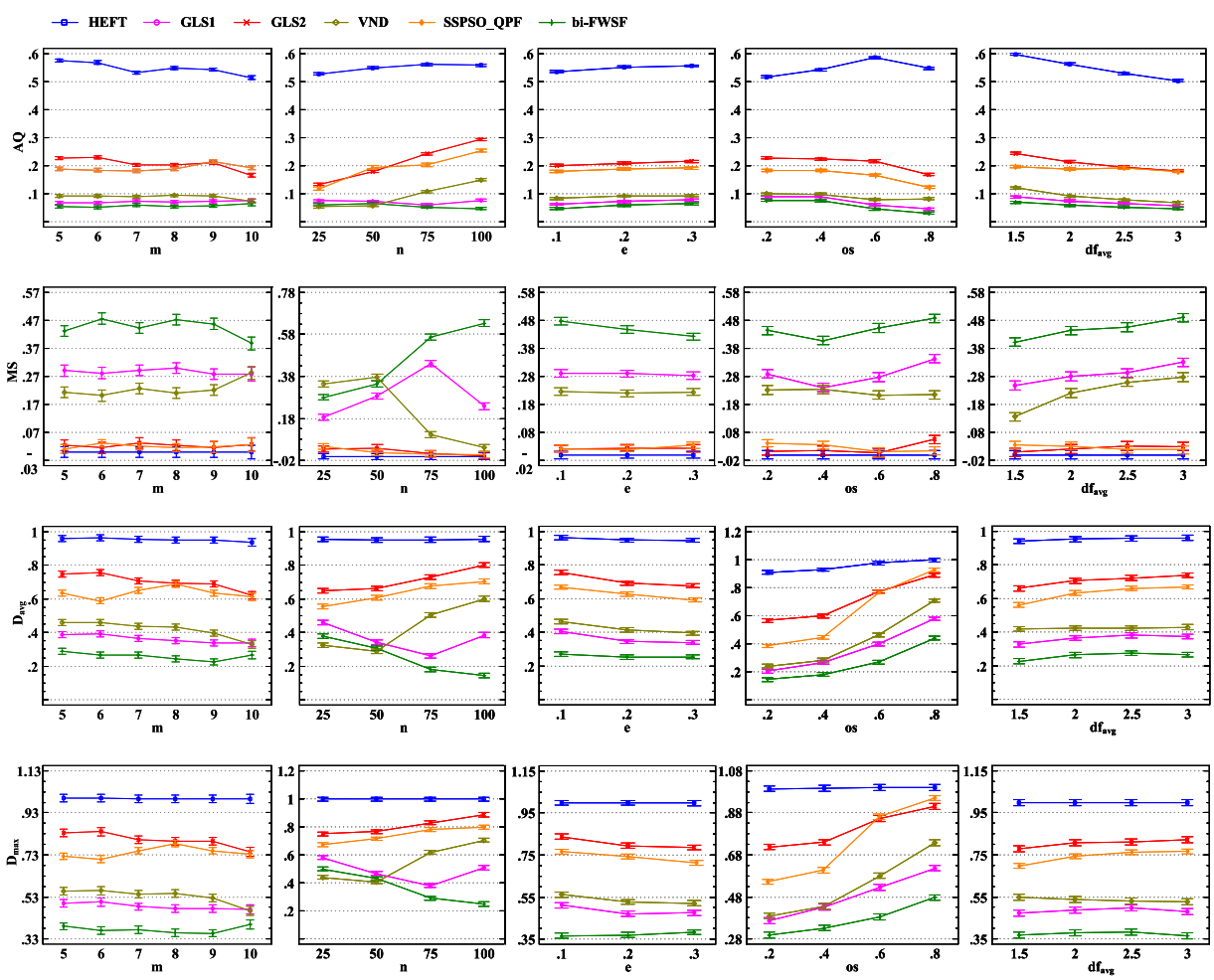



**Fig. 11.** Interactions between instance parameters and the compared algorithms with 95.0 % Tukey HSD intervals.

obtained by bi-FWSF, the computation time of bi-FWSF is affordable in practice. bi-FWSF is desirable for solving the problem considered.

## 6. Conclusion and future work

The paper addresses the challenge of scheduling compute-intensive workflows under temporal and deadline uncertainties in cloud environments. Three uncertain factors are investigated: the processing time of tasks, the transmission time of tasks, and the due date of workflows. By integrating fuzzy logic with multi-objective optimization, we developed a robust framework bi-FWSF that balances cost efficiency and user satisfaction. In bi-FWSF, the initial priorities of workflows are determined, and the initial solution is computed using the FSGM method. LL1/LL2 VM allocation strategies are developed in FSGM to arrange tasks to VMs. LL1 is set as the default choice for deadline-sensitive workflows. Then, a SAVNS heuristic with adaptive neighborhood structures is designed to explore the solution space starting from the initial solution. It outputs the best local solutions on the temporarily defined objectives. Finally, the fuzzy Pareto set is obtained and all fuzzy solutions are evaluated. To evaluate the performance of the proposal, five baseline algorithms are implemented and compared. The obtained non-dominated solutions of the compared algorithms are evaluated by multiple Pareto-based multi-objective metrics on aspects of solution quality, coverage of solution sets and approximation to the Pareto front. Based on experiments with varying fuzziness levels, we can conclude that bi-FWSF maintains stable performance with the highest solution quality, largest solution coverage, and closest approximation to the Pareto front. Although it is a little time-consuming, it is affordable in practice.

Although our study comprehensively evaluates the performance of the bi-FWSF algorithm through extensive simulations—incorporating realistic workflow structures, fuzzy temporal parameters, and hybrid cloud pricing models—we acknowledge that deploying the framework in a real cloud environment would further strengthen its practical applicability. Below, we discuss the steps we plan to take to address this limitation and enhance fuzzy-aware workflow scheduling in real cloud environments.

1. Deploying bi-FWSF on real cloud platforms can validate its performance under dynamic workloads and real cloud traces. This includes testing with actual workflow applications to measure metrics such as cost, satisfaction, and scalability in production environments. bi-FWSF has potential to be deployed on Apache Airavata, which provides a powerful framework to manage multiple computing resources and supports complex workflow management. Deploying the proposal on a real-world platform can offer practical and scientific benefits, bridging the gap between theoretical research and real-world impact. Full deployment requires further engineering, which will be one of our future works. We outline a six-step implementation plan for full deployment. (1) Develop a fuzzy evaluator module to assess the fuzzy processing time of workflows. (2) Construct the cloud environment and make it accessible to Apache Airavata. (3) Call the proposed algorithm to compute the set of non-dominated solutions based on fuzzy information, workflow structure, and available cloud resources. (4) Perform a decision module to select one solution from the set of non-dominated solutions. (5) Build a project in Apache Airavata. Define the workflow structure, the cloud resources, and the schedule plan in the project. (6) Run the project and monitor the execution of the workflows in real time.
2. The current model assumes static workflow arrivals. We plan to extend the framework to handle dynamic scenarios with stochastic task arrivals, VM failures, and network latency fluctuations. Techniques like reinforcement learning or online rescheduling could be explored to adapt to real-time uncertainties.
3. We will incorporate publicly available cloud workload traces (e.g., Google Cluster Data or Alibaba Cluster Trace) to benchmark bi-FWSF against state-of-the-art schedulers under empirically observed conditions.

## CRediT authorship contribution statement

**Jie Zhu:** Writing – original draft, Methodology, Investigation. **Fengmei Liu:** Validation, Software, Formal analysis. **Jingzhe Sun:** Validation, Software. **Haiping Huang:** Writing – review & editing, Supervision, Funding acquisition.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgment

This work is sponsored by the Major Re-search Plan of the National Natural Science Foundation of China (No. 92467202), Jiangsu Province Universities Natural Science Research Major Project (Grant No. 23KJA520010), the Jiangsu Postgraduate Practical Innovation Program (Grant No. KYCX24_1229), and the State Key Laboratory of Computer Architecture (ICT, CAS) (Grant No. CARCHA202107).

## Data availability

I have shared the link to my data and code in the revised manuscript.

## References

[1] J. Darbya, M.B. Sanchez, S. Bew, I. Loram, P. Butler, the development of a video retrieval system using a clinician-led approach, Expert Syst. Appl. 142 (15) (2020) 112992.
[2] R. Xie, K. Ibrahim, B. Shahriar, A. Mohammed, Fast and peer-to-peer vital signal learning system for cloud-based healthcare, Futur. Gener. Comput. Syst. 88 (2018) 220–233.
[3] J. Zhu, X. Li, R. Ruiz, X. Xu, Scheduling stochastic multi-stage jobs to elastic hybrid cloud resources, IEEE Trans. Parallel Distrib. Syst. 29 (6) (2018) 1401–1415.
[4] M.D. de Assuncao, C.H. Cardonha, M.A. Netto, R.L. Cunha, Impact of user patience on auto-scaling resource capacity for cloud services, Futur. Gener. Comput. Syst. 55 (2016) 41–50.
[5] J. Zhu, X. Li, R. Ruiz, L. Wei, H. Huang, Y. Zomayaalbert, Scheduling periodical multi-stage jobs with fuzziness to elastic cloud resources, IEEE Trans. Parallel Distrib. Syst. 31 (12) (2020) 2819–2833.
[6] R. Chen, F. Li, D. Luna, I. Ranawaka, F. Song, S. Pamidighantam, X. Liang, Y. Liang, Asynchronous modeling workflows in Cyberwater with on-demand HPC/cloud access, Futur. Gener. Comput. Syst. 159 (2024) 307–322.
[7] P. Pinto, J. Bispo, J.M.P. Cardoso, J.G. Barbosa, D. Gadioli, G. Palermo, Pegasus: performance engineering for software applications targeting HPC systems, IEEE Trans. Softw. Eng. 48 (3) (2022) 732–754.
[8] M. Wilde, M. Hategan, J.M. Wozniak, B. Clifford, D.S. Katz, I. Foster, Swift and accurate mobility-aware QoS forecasting for mobile edge environments, in: IEEE Transactions on Services Computing, Early Access, 2024, pp. 1–14, https://doi.org/10.1109/TSC.2024.3417339
[9] M. Wilde, M. Hategan, J.M. Wozniak, B. Clifford, D.S. Katz, I. Foster, Deadline-constrained security-aware workflow scheduling in hybrid cloud architecture, Futur. Gener. Comput. Syst. 162 (2025) 107466, https://doi.org/10.1016/j.future.2024.07.044
[10] L. Yang, Y. Xia, L. Ye, R. Gao, Y. Zhan, A fully hybrid algorithm for deadline constrained workflow scheduling in clouds, IEEE Trans. Cloud Comput. 11 (3) (2023) 3197–3210.
[11] L. Ye, Y. Xia, L. Yang, Y. Zhan, Dynamic scheduling stochastic multiworkflows with deadline constraints in clouds, IEEE Trans. Autom. Sci. Eng. 20 (4) (2023) 2594–2606.
[12] L. Ye, L. Yang, Y. Xia, X. Zhao, A cost-driven intelligence scheduling approach for deadline-constrained IoT workflow applications in cloud computing, IEEE Internet Things J. 11 (9) (2024) 16033–16047.
[13] H. Taheri, S. Abrishami, M. Naghibzadeh, A cloud broker for executing deadline-constrained periodic scientific workflows, IEEE Trans. Serv. Comput. 16 (5) (2023) 3089–3100.
[14] K. Rajput, X. Li, J. Zhang, A. Lakhan, A novel scheduling approach for Spark workflow tasks with deadline and uncertain performance in multi-cloud networks, IEEE Trans. Cloud Comput. 12 (4) (2024) 1145–1157.
[15] X. Li, W. Yu, R. Ruiz, J. Zhu, Energy-aware cloud workflow applications scheduling with geo-distributed data, IEEE Trans. Serv. Comput. 15 (2) (2022) 891–903.
[16] Z. Sun, B. Zhang, C. Gu, R. Xie, B. Qian, H. Huang, ET2FA: a hybrid heuristic algorithm for deadline-constrained workflow scheduling in cloud, IEEE Trans. Serv. Comput. 16 (3) (2023) 1807–1821.

[17] C. Lin, N. Mahmoudi, C. Fan, H. Khazaei, Fine-grained performance and cost modeling and optimization for FaaS applicationsd, IEEE Trans. Parallel Distrib. Syst. 34 (1) (2023) 180–194.
[18] Q. Wu, M. Zhou, Q. Zhu, Y. Xia, J. Wen, MOELS: multiobjective evolutionary list scheduling for cloud workflows, IEEE Trans. Autom. Sci. Eng. 17 (1) (2020) 166–176.
[19] S. Qin, D. Pi, Z. Shao, Y. Xu, A cluster-based cooperative co-evolutionary algorithm for multiobjective workflow scheduling in a cloud environment, IEEE Trans. Autom. Sci. Eng. 20 (3) (2023) 1648–1662.
[20] H. Li, B. Wang, Y. Yuan, M. Zhou, Y. Fan, Y. Xia, Scoring and dynamic hierarchy-based NSGA-ii for multiobjective workflow scheduling in the cloud, IEEE Trans. Autom. Sci. Eng. 19 (2) (2022) 982–993.
[21] D. Wu, X. Wang, X. Wang, M. Huang, R. Zeng, K. Yang, Multi-objective optimization-based workflow scheduling for applications with data locality and deadline constraints in geo-distributed clouds, Futur. Gener. Comput. Syst. 157 (2024) 485–498.
[22] Y. Guo, B. Liu, W. Lin, X. Ye, J.Z. Wang, Dynamic neighborhood grouping-based multi-objective scheduling algorithm for workflow in hybrid cloud, Futur. Gener. Comput. Syst. 166 (2025) 107633.
[23] L. Zhu, B. Li, L. Tan, A digital twin-based multi-objective optimized task offloading and scheduling scheme for vehicular edge networks, Futur. Gener. Comput. Syst. 163 (2025) 107517.
[24] Z. Zhang, M. Zhao, H. Wang, Z. Cui, W. Zhang, An efficient interval many-objective evolutionary algorithm for cloud task scheduling problem under uncertainty, Inf. Sci. 583 (2022) 56–72.
[25] N. Rizvi, D. Ramesh, P.C. Srinivasa, K. Mondal, Intelligent salp swarm scheduler with fitness based quasi-reflection method for scientific workflows in hybrid cloud-fog environment, IEEE Trans. Autom. Sci. Eng. 20 (2) (2023) 862–877.
[26] X. Chen, C. Lin, B. Lin, An intelligent workflow scheduling scheme for complex network robustness in fuzzy edge-cloud environments, IEEE Trans. Netw. Sci. Eng. 11 (1) (2024) 1106–1123.
[27] E. Demeulemeester, M. Vanhoucke, W. Herroelen, rangen: a random network generator for activity-on-the-node networks, J. Sched. 6 (1) (2003) 17–38.
[28] B. Li, L. Wang, A hybrid quantum-inspired genetic algorithm for multiobjective flow shop scheduling, IEEE Trans. Cybern. 37 (3) (2007) 576–591.
[29] E. Demeulemeester, M. Vanhoucke, W. Herroelen, rangen: a random network generator for activity-on-the-node networks, J. Sched. 6 (1) (2003) 17–38.
[30] X. Zhou, G. Zhang, J. Sun, J. Zhou, T. Wei, S. Hu, Minimizing cost and makespan for workflow scheduling in cloud using fuzzy dominance sort based HEFT, Futur. Gener. Comput. Syst. 93 (2019) 278–289.
[31] J. Zhu, X. Li, Scheduling for multi-stage applications with scalable virtual resources in cloud computing, Int. J. Mach. Learn. Cybern. 8 (5) (2016) 1633–1641.
[32] F. Anka, N. Agaoglu, S. Nematzadeh, M. Torkamanian - Afshar, F. Soleimanian Gharehchopogh, Advances in artificial rabbits optimization: a comprehensive review, Arch. Comput. Methods Eng. 32 (4) (2024) 2113–2148.
[33] F.S. Gharehchopogh, S. Ghafouri, M. Namazi, et al., Advances in manta ray foraging optimization: a comprehensive survey, J. Bionic Eng. 21 (2) (2024) 953–990.
[34] F.S. Gharehchopogh, B. Abdollahzadeh, B. Arasteh, An improved farmland fertility algorithm with hyper - heuristic approach for solving travelling salesman problem, Comput. Model. Eng. Sci. 135 (3) (2022) 1981–2006.
[35] A.O. Abdulsalami, M.A. Elaziz, F.S. Gharehchopogh, A.T. Salawudeen, S. Xiong, An improved heterogeneous comprehensive learning symbiotic organism search for optimization problems, Knowl.-Based Syst. 285 (2024) 111351.
[36] J. Wu, Y.-G. Wang, K. Burrage, Y.-C. Tian, B. Lawson, Z. Ding, An improved firefly algorithm for global continuous optimization problems, Expert Syst. Appl. 149 (2020) 113340.